

Project 2

Marc Hernandez, Tilboon Elberier

```
In [70]: # import functions needed for problems 1-18
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.decomposition import TruncatedSVD
from sklearn.decomposition import NMF
from sklearn.cluster import KMeans
from sklearn.metrics.cluster import contingency_matrix, homogeneity_score, v_measure
from sklearn.cluster import AgglomerativeClustering

from scipy import stats
from scipy.optimize import linear_sum_assignment
import importlib
import plotmat
import hdbscan

import warnings
warnings.filterwarnings("ignore")
```

```
In [2]: # Import newsgroup data for TFIDF Transformer
cata = ['comp.graphics', 'comp.os.ms-windows.misc', 'comp.sys.ibm.pc.hardware', 'co
        'rec.autos', 'rec.motorcycles', 'rec.sport.baseball', 'rec.sport.hockey']
newsgroups train = fetch_20newsgroups(subset='all', categories=cata, shuffle=True,
```

QUESTION 1: Report the dimensions of the TF-IDF matrix you obtain.

```
In [3]: # Find the Shape of the
countVectorizer = CountVectorizer(min_df=3, stop_words='english')
tfidf_transformer = TfidfTransformer()
newsgroups_counts = countVectorizer.fit_transform(newsgroups_train.data)
newsgroups_tfidf = tfidf_transformer.fit_transform(newsgroups_counts)
print("ANSWER QUESTION 1: The shape of the TF-IDF matrix is ", newsgroups_tfidf.shape)
```

ANSWER QUESTION 1: The shape of the TF-IDF matrix is (7882, 27768)

QUESTION 2: Report the contingency table of your clustering result. You may use the provided plotmat.py to visualize the matrix. Does the contingency matrix have to be square-shaped?

```
In [4]: km = KMeans(n_clusters=2, random_state=0, max_iter=1000, n_init=30)
y_pred = km.fit_predict(newsgroups_tfidf)
y_true = [int(i/4) for i in newsgroups_train.target]
```

```

cluster_class_names=['Class 1', 'Class 2']
actual_class_names=['Cluster 1', 'Cluster 2']

cm = contingency_matrix(y_true, y_pred)
print("Contingency table: \n", cm)
plt.imshow(cm, cmap=plt.cm.Blues)
plt.colorbar()
plt.xticks(range(len(actual_class_names)), actual_class_names)
plt.yticks(range(len(actual_class_names)), cluster_class_names)

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        if np.argmax(cm) == i*cm.shape[1]+j:
            plt.text(j, i, cm[i, j], horizontalalignment="center", color='w')
        else:
            plt.text(j, i, cm[i, j], horizontalalignment="center", color='k')

plt.tight_layout()
plt.xlabel('Ground Truth')
plt.ylabel('Predicted Label')
plt.title('Contingency Table', fontweight='bold')
plt.show()

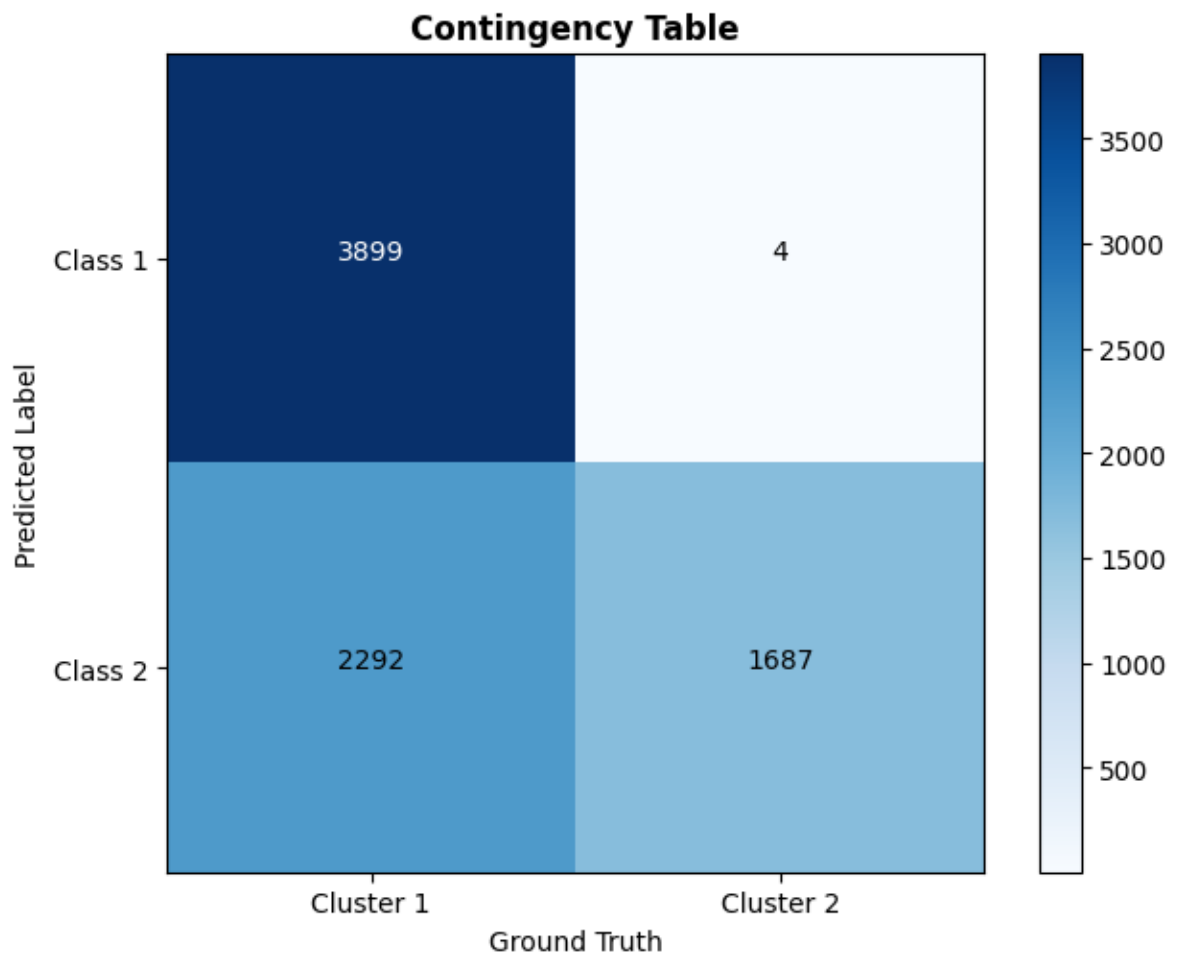
```

Contingency table:

```

[[3899    4]
 [2292 1687]]

```



No, the contingency matrix doesn't have to be square shaped because the matrix itself represents the frequency of occurrences between our predictions at the group truth label. The shape of the contingency matrix could be rectangular.

QUESTION 3: Report the 5 clustering measures explained in the introduction for K- means clustering.

```
In [5]: print("Below are the 5 clustering Evaluation Metrics described within the project 2")
def get_metrics(y_true, y_pred):
    diction = {}
    metrics = [homogeneity_score, completeness_score, v_measure_score, adjusted_rand_score]

    for i in metrics:
        diction[i.__name__] = i(y_true, y_pred)

    df = pd.DataFrame(diction, index=[0]).T
    df.reset_index(inplace=True)
    df.rename(columns={'index': 'Evaluation', 0: 'Result'}, inplace=True)

    return df

get_metrics(y_true, y_pred)
```

Below are the 5 clustering Evaluation Metrics described within the project 2 documentation

Out[5]:

	Evaluation	Result
0	homogeneity_score	0.247946
1	completeness_score	0.330542
2	v_measure_score	0.283348
3	adjusted_rand_score	0.174133
4	adjusted_mutual_info_score	0.283273

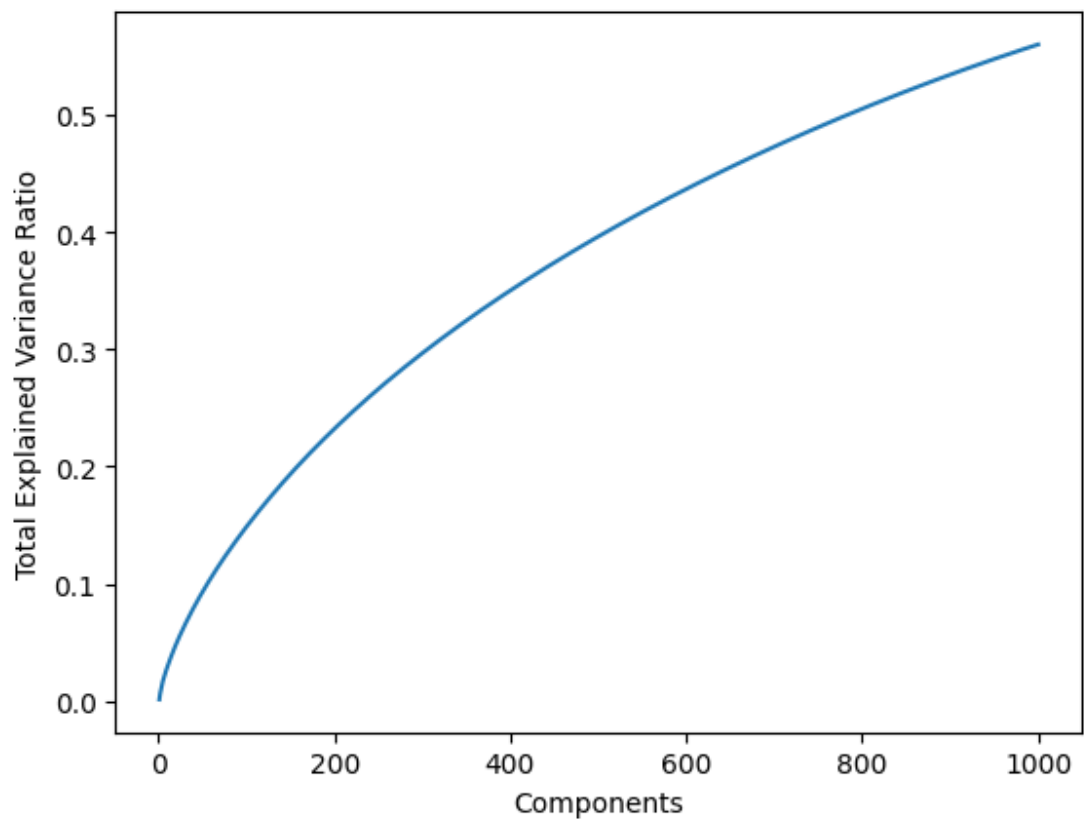
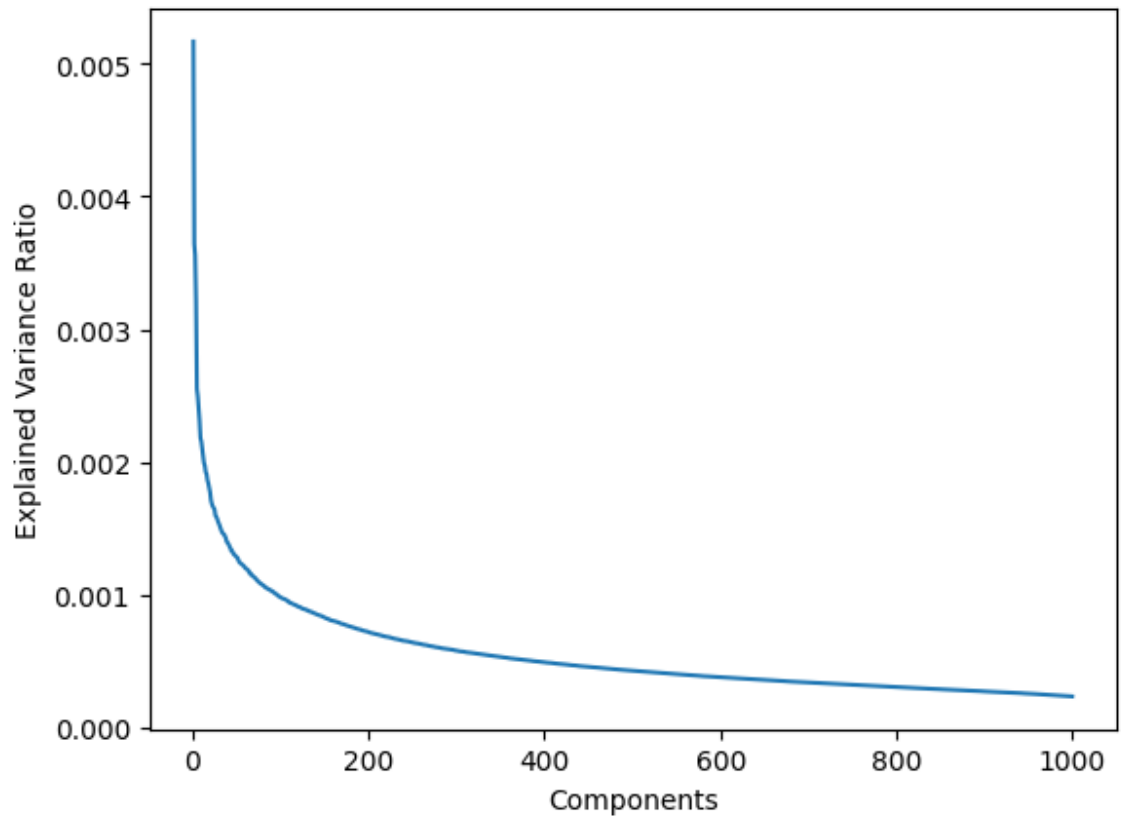
QUESTION 4: Report the plot of the percentage of variance that the top r principle components retain v.s. r , for $r = 1$ to 1000.

```
In [6]: svd = TruncatedSVD(n_components=1000, random_state=42)
X_svd = svd.fit_transform(newsgroups_tfidf)

plt.figure()
plt.plot(np.arange(1000) + 1, sorted(svd.explained_variance_ratio_, reverse=True))
plt.xlabel("Components")
plt.ylabel("Explained Variance Ratio")

plt.figure()
plt.plot(np.arange(1000) + 1, np.cumsum(svd.explained_variance_ratio_))
plt.xlabel("Components")
plt.ylabel("Total Explained Variance Ratio")

Out[6]: Text(0, 0.5, 'Total Explained Variance Ratio')
```



QUESTION 5: Let r be the dimension that we want to reduce the data to (i.e. n components). Try $r = 1 - 10, 20, 50, 100, 300$, and plot the 5 measure scores v.s. r for both SVD and NMF.

Report a good choice of r for SVD and NMF respectively.

Note: In the choice of r , there is a trade-off between the information preservation, and better performance of k -means in lower dimensions.

```
In [7]: components = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 50, 100, 300]
km = KMeans(n_clusters=2, random_state=0, max_iter=1000, n_init=30)

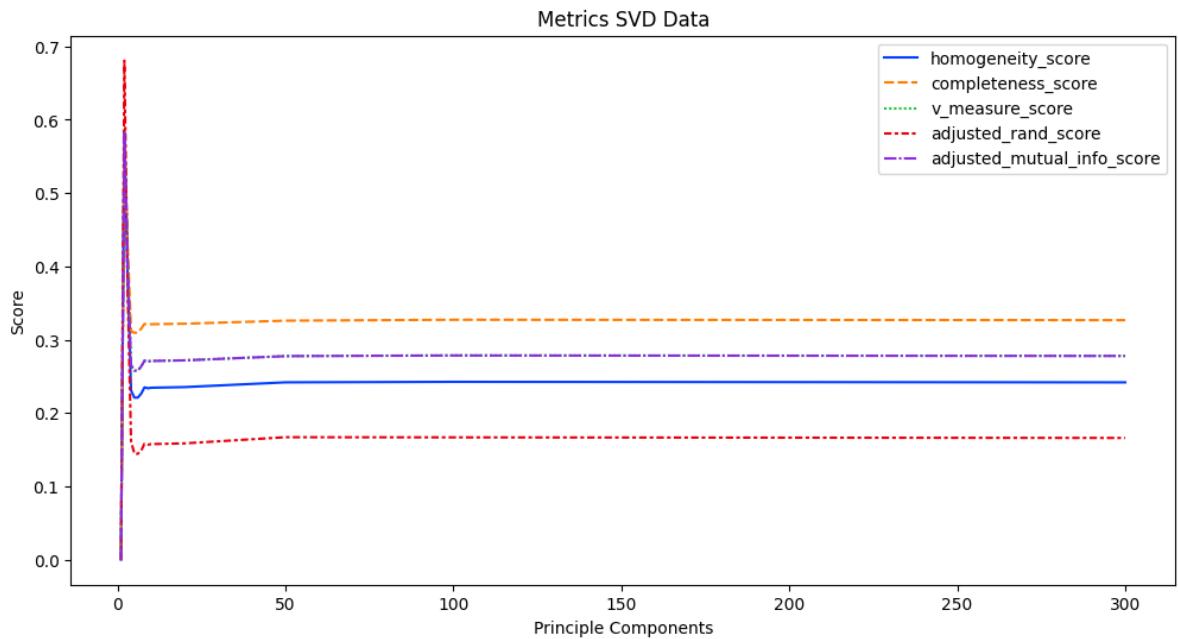
result = []
for r in components:
    svd = TruncatedSVD(n_components=r, random_state=42).fit_transform(newsgroups_tf)
    preds = km.fit_predict(svd)
    result.append(get_metrics(y_true, preds)['Result'].tolist())

metrics = get_metrics(y_true, preds)['Evaluation'].tolist()
metrics_df = pd.DataFrame(result, columns=metrics, index=components)
metrics_df
```

```
Out[7]:
```

	homogeneity_score	completeness_score	v_measure_score	adjusted_rand_score	adjusted_mutual_info_score
1	0.000284	0.000289	0.000286	0.000317	0.000286
2	0.583085	0.585240	0.584160	0.680448	0.584160
3	0.382045	0.425804	0.402739	0.364320	0.402739
4	0.230775	0.311849	0.265255	0.161654	0.265255
5	0.221171	0.309568	0.258008	0.144576	0.258008
6	0.221171	0.309568	0.258008	0.144576	0.258008
7	0.226504	0.315106	0.263557	0.148658	0.263557
8	0.235158	0.321620	0.271676	0.158405	0.271676
9	0.233915	0.320684	0.270512	0.156993	0.270512
10	0.234625	0.321219	0.271177	0.157799	0.271177
20	0.235514	0.321888	0.272009	0.158809	0.272009
50	0.241986	0.326035	0.277792	0.167212	0.277792
100	0.242668	0.327276	0.278692	0.167004	0.278692
300	0.241949	0.326734	0.278022	0.166176	0.278022

```
In [8]: plt.figure(figsize=(12, 6))
sns.set_palette("bright")
g = sns.lineplot(data=metrics_df)
g.set_xlabel('Principle Components')
g.set_ylabel('Score')
g.set_title('Metrics SVD Data')
plt.show()
```



```
In [9]: result = []
for r in components:
    nmf = NMF(n_components=r, random_state=42).fit_transform(newsgroups_tfidf)
    preds = km.fit_predict(nmf)
    result.append(get_metrics(y_true, preds)['Result'].tolist())

metrics = get_metrics(y_true, preds)['Evaluation'].tolist()
metrics_df = pd.DataFrame(result, columns=metrics, index=components)
metrics_df
```

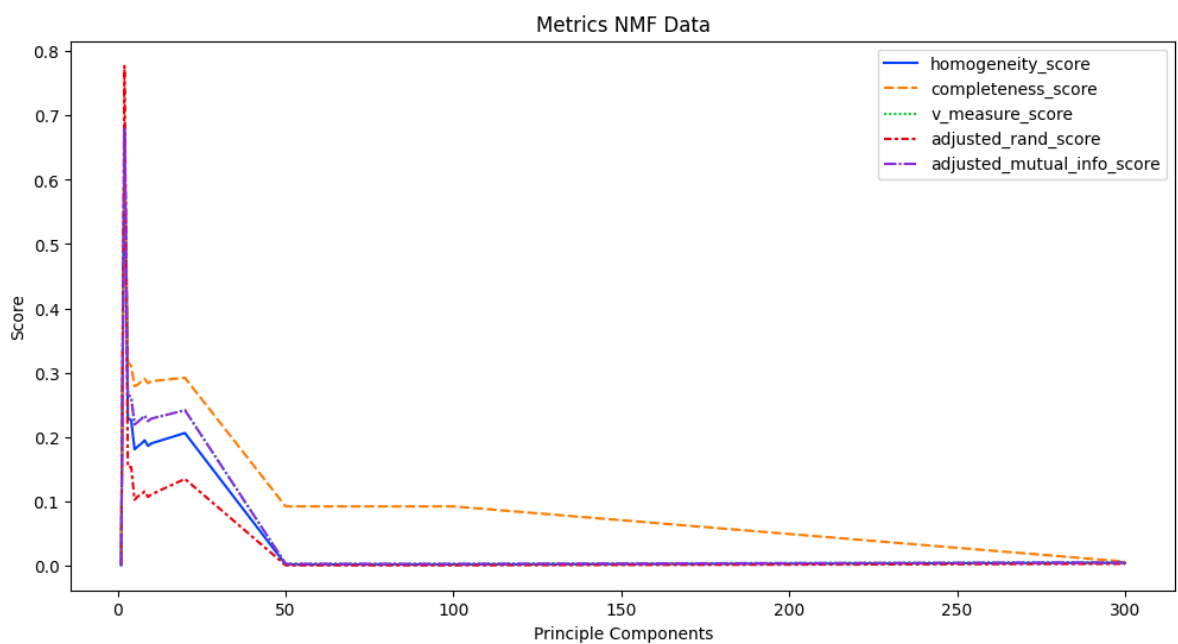
```
c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\decomposition\_nmf.py:1742: ConvergenceWarning: Maximum number of iterations 200 reached.
Increase it to improve convergence.
  warnings.warn(
c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\decomposition\_nmf.py:1742: ConvergenceWarning: Maximum number of iterations 200 reached.
Increase it to improve convergence.
  warnings.warn(
c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\decomposition\_nmf.py:1742: ConvergenceWarning: Maximum number of iterations 200 reached.
Increase it to improve convergence.
  warnings.warn(
```

Out[9]:	homogeneity_score	completeness_score	v_measure_score	adjusted_rand_score	adjusted_mutual_info_score
1	0.000284	0.000289	0.000286	0.000317	0.000284
2	0.679048	0.680132	0.679590	0.777018	0.679048
3	0.229343	0.316484	0.265957	0.152797	0.229343
4	0.225779	0.310382	0.261406	0.152996	0.225779
5	0.180631	0.278709	0.219199	0.101956	0.180631
6	0.185311	0.281480	0.223489	0.107376	0.185311
7	0.189325	0.286194	0.227893	0.109886	0.189325
8	0.194655	0.290243	0.233028	0.115338	0.194655
9	0.185850	0.283550	0.224533	0.106380	0.185850
10	0.189823	0.286573	0.228373	0.110392	0.189823
20	0.205946	0.291986	0.241533	0.134526	0.205946
50	0.001288	0.091729	0.002540	0.000055	0.001288
100	0.001288	0.091729	0.002540	0.000055	0.001288
300	0.003918	0.005839	0.004689	0.002361	0.003918

```

In [10]: plt.figure(figsize=(12, 6))
sns.set_palette("bright")
g = sns.lineplot(data=metrics_df)
g.set_xlabel('Principle Components')
g.set_ylabel('Score')
g.set_title('Metrics NMF Data')
plt.show()

```



QUESTION 6: How do you explain the non-monotonic behavior of the measures as r increases?

From our graphs we can observe that the influence of the number of components is non-monotonic, as the resulting score does not increase consistently as the number of components also increases. This is likely due to the curse of dimensionality as, K-means relies on the Euclidean Distance to calculate distances between points and nearby clusters, and as we continue to increase the number of cluster groups (i.e. as the number of components increases) then it will be more difficult for our model to properly capture the information provided by the data.

QUESTION 7: Are these measures on average better than those computed in Question 3?

Yes, on average the measurements observed using SVD average do perform better, however, NMF is close and could be argued that it doesn't perform better than the baseline implementation in question 3

QUESTION 8: Visualize the clustering results for:

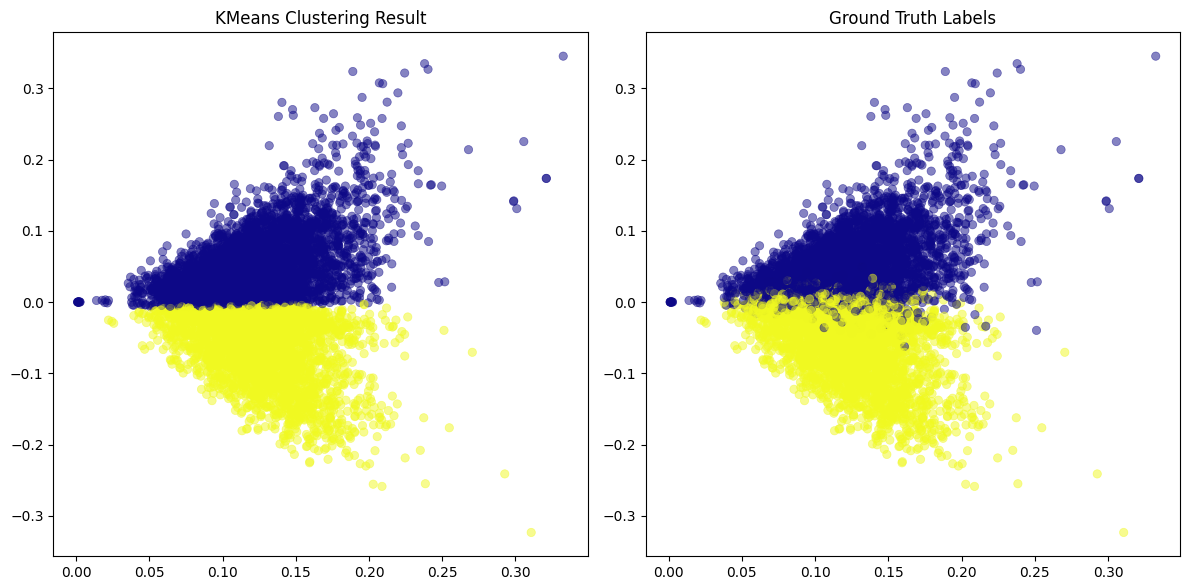
- SVD with your optimal choice of r for K-Means clustering;
- NMF with your choice of r for K-Means clustering.

To recap, you can accomplish this by first creating the dense representations and then once again projecting these representations into a 2-D plane for visualization.

```
In [11]: # Best number of components was  $r = 2$ 
svd = TruncatedSVD(n_components=2, random_state=42).fit_transform(newsgroups_tfidf)
svd_pred = km.fit_predict(svd)

plt.gcf().clear()
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.scatter(svd[:,0], svd[:,1], c=svd_pred, linewidth=0.5, cmap="plasma", alpha=0.5)
plt.title('KMeans Clustering Result')
plt.subplot(1,2,2)
plt.scatter(svd[:,0], svd[:,1], c=y_true, linewidth=0.5, cmap="plasma", alpha=0.5)
plt.title('Ground Truth Labels')
plt.tight_layout()
plt.show()
```

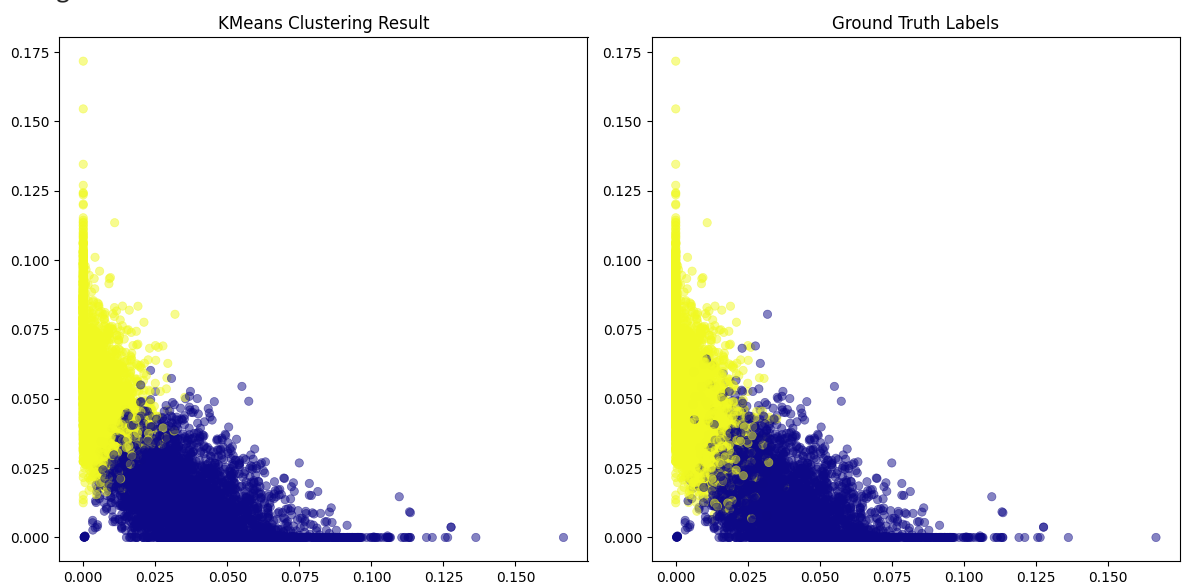
<Figure size 640x480 with 0 Axes>



```
In [12]: nmf = NMF(n_components=2, random_state=42).fit_transform(newsgroups_tfidf)
nmf_pred = km.fit_predict(nmf)

plt.gcf().clear()
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.scatter(nmf[:,0], nmf[:,1], c=nmf_pred, cmap="plasma",linewidth=0.5, alpha=0.5)
plt.title('KMeans Clustering Result')
plt.subplot(1,2,2)
plt.scatter(nmf[:,0], nmf[:,1], c=y_true, cmap="plasma", linewidth=0.5, alpha=0.5)
plt.title('Ground Truth Labels')
plt.tight_layout()
plt.show()
```

<Figure size 640x480 with 0 Axes>



QUESTION 9: What do you observe in the visualization? How are the data points of the two classes distributed? Is distribution of the data ideal for K-Means clustering?

Based on the plots depicted above, one can say that K-means will likely not be the most optimal model for this dataset. Primarily because of the non-isotropic nature of the formed clusters as well as their frequent overlap in both the SVD and NMF graphs. Because of these two issues, this is not an ideal distribution for the use of K-Means clustering.

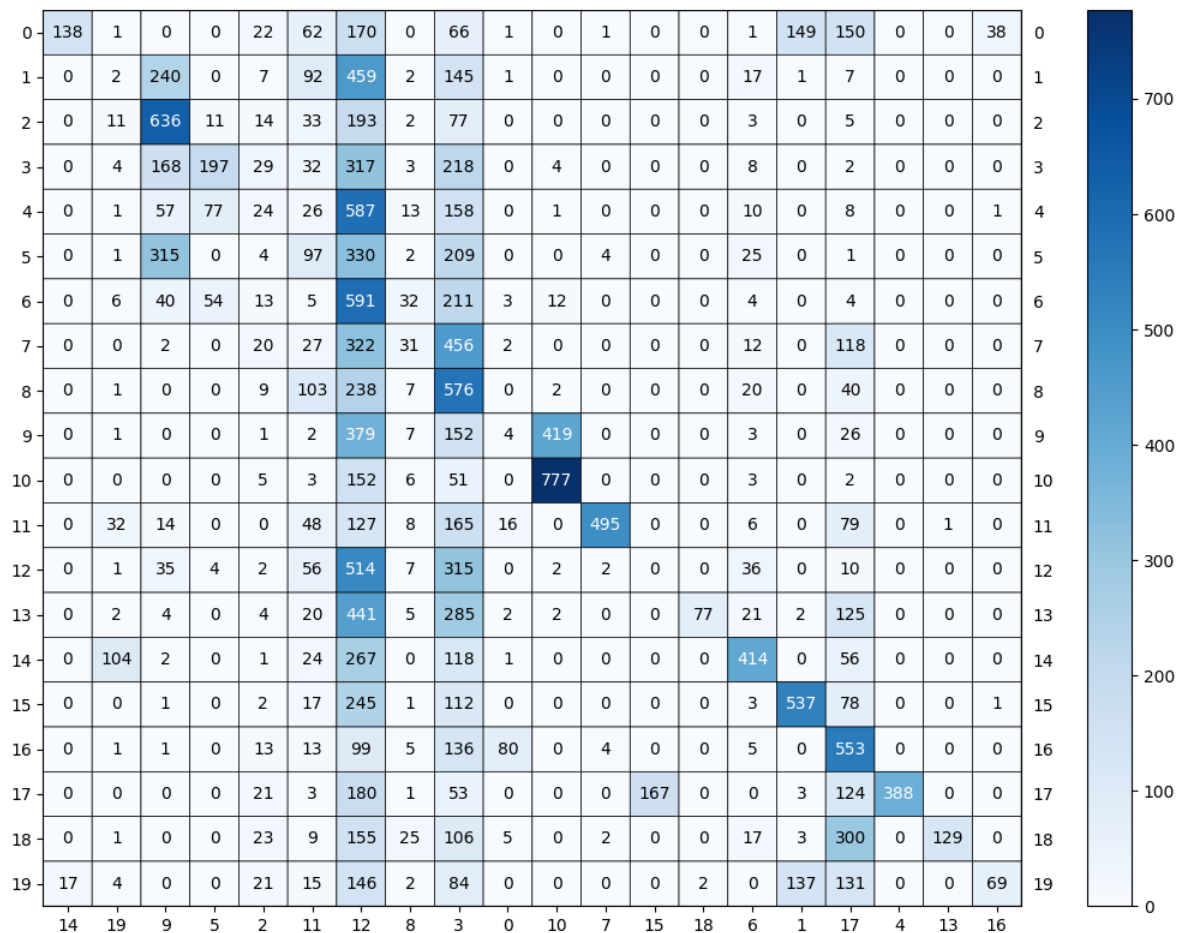
QUESTION 10: Load documents with the same configuration as in Question 1, but for ALL 20 categories. Construct the TF-IDF matrix, reduce its dimensionality using BOTH NMF and SVD (specify settings you choose and why), and perform K-Means clustering with $k=20$. Visualize the contingency matrix and report the five clustering metrics (DO BOTH NMF AND SVD).

There is a mismatch between cluster labels and class labels. For example, the cluster #3 may correspond to the class #8. As a result, the high-value entries of the 20×20 contingency matrix can be scattered around, making it messy to inspect, even if the clustering result is not bad.

```
In [98]: data_full = fetch_20newsgroups(subset='all', shuffle=True, random_state=42)
```

```
In [99]: count_full = CountVectorizer(min_df=3, stop_words='english')
tfidf_full = TfidfTransformer()
km = KMeans(n_clusters=20, random_state=0, max_iter=1000, n_init=30)
full_counts = count_full.fit_transform(data_full.data)
tfidf_preds = tfidf_full.fit_transform(full_counts)

svd = TruncatedSVD(n_components=20, random_state=42).fit_transform(tfidf_preds)
svd_pred = km.fit_predict(svd)
cm = contingency_matrix(data_full.target, svd_pred)
rows, cols = linear_sum_assignment(cm, maximize=True)
reordered_mat = cm[np.ix_(rows, cols)]
plotmat.plot_mat(reordered_mat, xticklabels=cols, yticklabels=rows, size=(10,8))
```



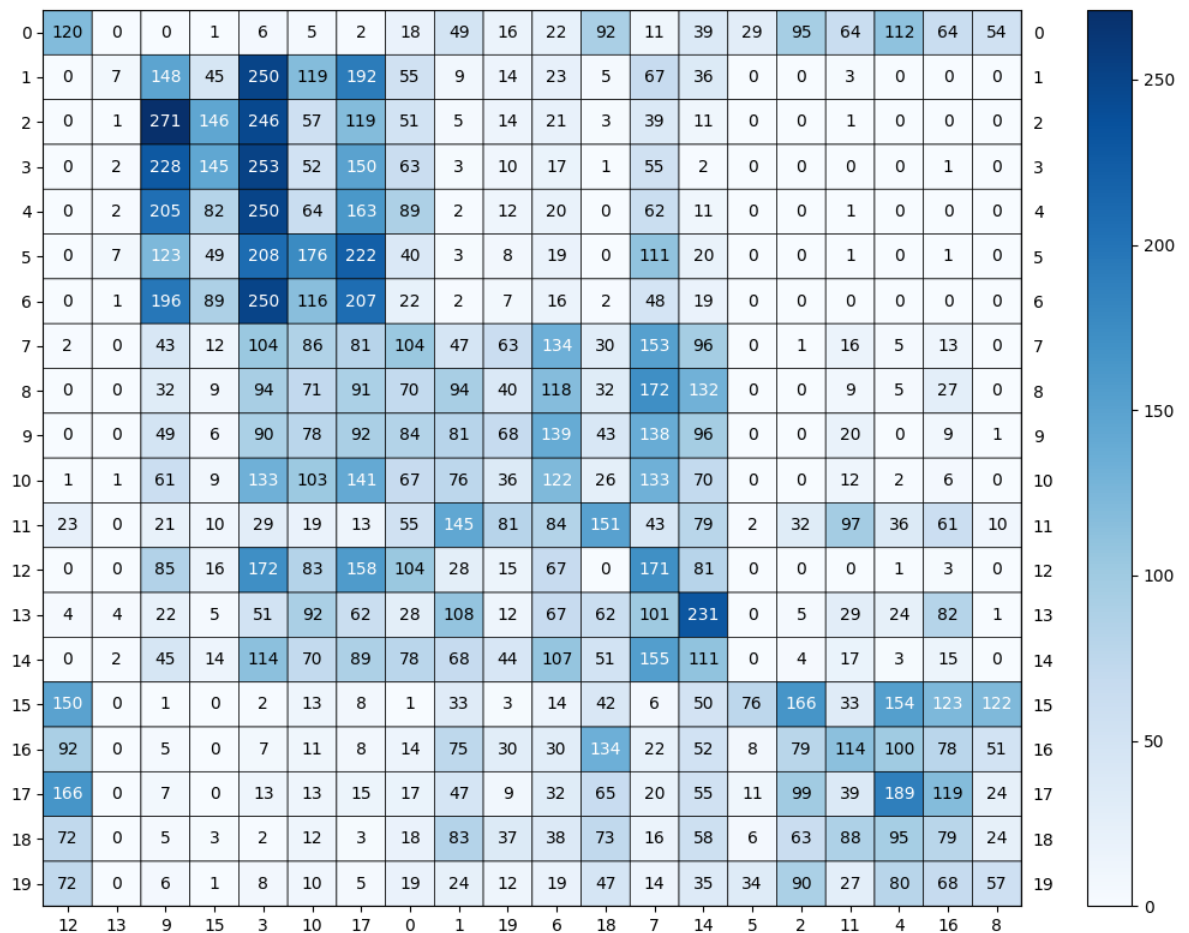
```
In [47]: get_metrics(data_full.target, y_pred)
```

```
Out[47]:
```

	Evaluation	Result
0	homogeneity_score	0.287995
1	completeness_score	0.383499
2	v_measure_score	0.328955
3	adjusted_rand_score	0.093060
4	adjusted_mutual_info_score	0.326445

```
In [100... count_full = CountVectorizer(min_df=3, stop_words='english')
tfidf_full = TfidfTransformer()
km = KMeans(n_clusters=20, random_state=0, max_iter=1000, n_init=30)
full_counts = count_full.fit_transform(data_full.data)
tfidf_preds = tfidf_full.fit_transform(full_counts)

nmf = NMF(n_components=2, random_state=42).fit_transform(tfidf_preds)
nmf_pred = km.fit_predict(nmf)
cm = contingency_matrix(data_full.target, nmf_pred)
rows, cols = linear_sum_assignment(cm, maximize=True)
reordered_mat = cm[np.ix_(rows, cols)]
plotmat.plot_mat(reordered_mat, xticklabels=cols, yticklabels=rows, size=(10,8))
```



```
In [46]: get_metrics(data_full.target, nmf_pred)
```

```
Out[46]:
```

	Evaluation	Result
0	homogeneity_score	0.169491
1	completeness_score	0.179924
2	v_measure_score	0.174552
3	adjusted_rand_score	0.051011
4	adjusted_mutual_info_score	0.171780

QUESTION 11: Reduce the dimension of your dataset with UMAP. Consider the following settings: n components = [5, 20, 200], metric = "cosine" vs. "euclidean". If "cosine" metric fails, please look at the FAQ at the end of this spec.

Report the permuted contingency matrix and the five clustering evaluation metrics for the different combinations (6 combinations).

```
In [63]: import umap.umap_ as umap

components = [5, 20, 200]
metric = ["cosine", "euclidean"]
```

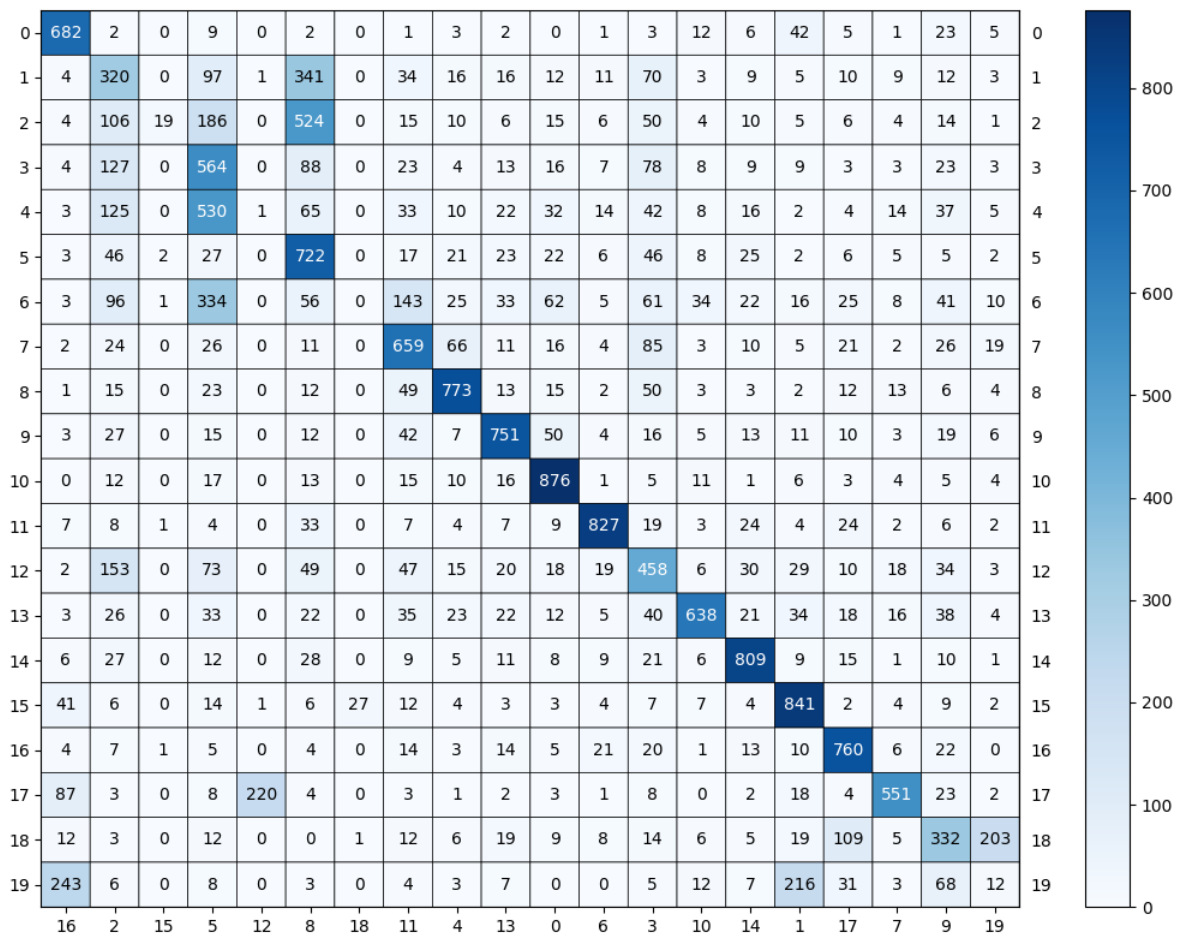
```

for n in components:
    for i in metric:
        umap_full = umap.UMAP(n_components=n, metric=i).fit_transform(tfidf_preds)
        km = KMeans(n_clusters=20, random_state=0, max_iter=1000, n_init=30)
        y_pred = km.fit_predict(umap_full)

        cm = contingency_matrix(data_full.target, y_pred)
        rows, cols = linear_sum_assignment(cm, maximize=True)
        reordered_mat = cm[np.ix_(rows, cols)]
        plotmat.plot_mat(reordered_mat, xticklabels=cols, yticklabels=rows, size=(1

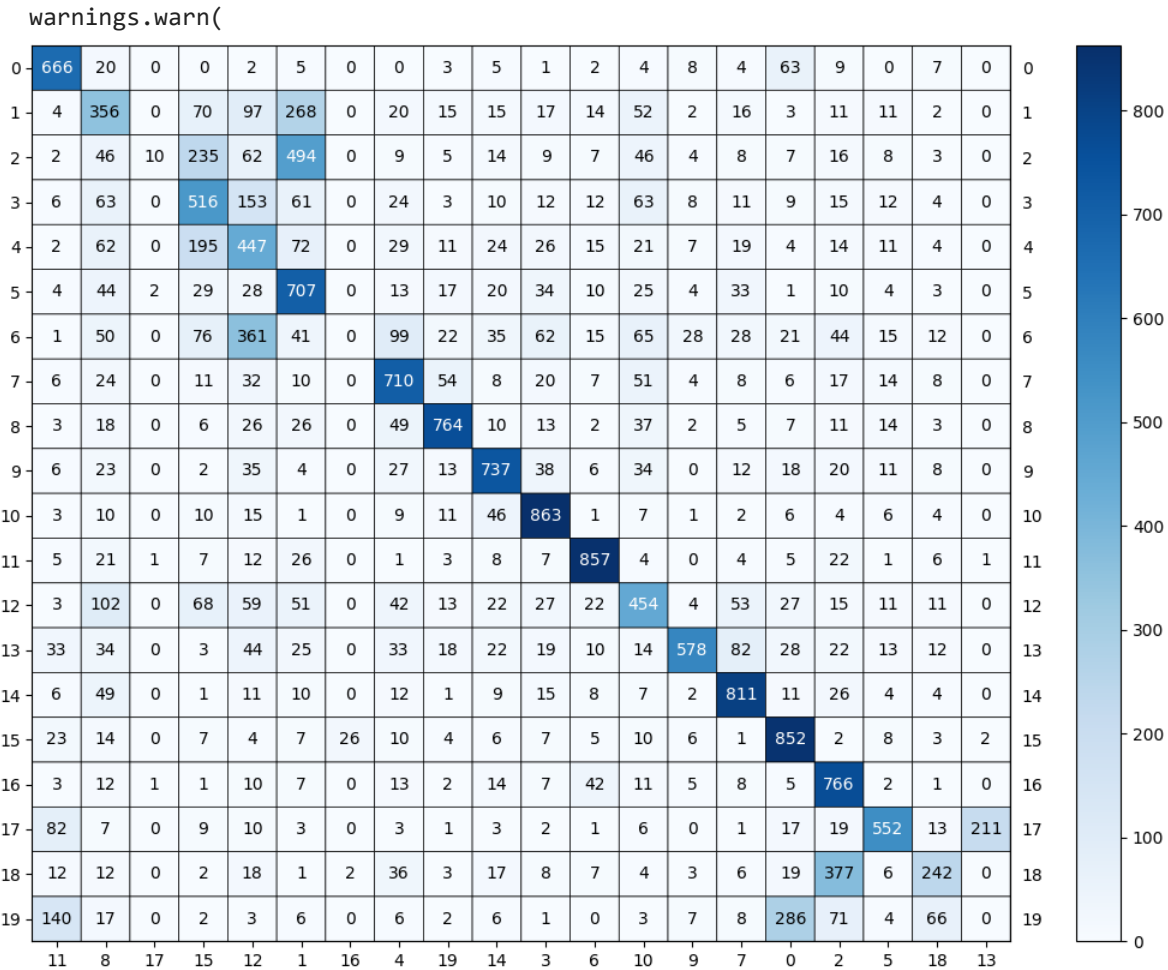
print("Number of Components: ", n, "Evaluating Metric: ", i)
print("Homogeneity : %0.3f" % homogeneity_score(data_full.target, y_pred))
print("Completeness : %0.3f" % completeness_score(data_full.target, y_pred))
print("V-measure : %0.3f" % v_measure_score(data_full.target, y_pred))
print("Adjusted Rand-Index : %0.3f" % adjusted_rand_score(data_full.target, y
print("Adjusted Mutual Information Score : %0.3f" % adjusted_mutual_info_scor

```



Number of Components: 5 Evaluating Metric: cosine
 Homogeneity : 0.494
 Completeness : 0.526
 V-measure : 0.510
 Adjusted Rand-Index : 0.389
 Adjusted Mutual Information Score : 0.508

c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\utils\deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.



Number of Components: 5 Evaluating Metric: euclidean

Homogeneity : 0.492

Completeness : 0.520

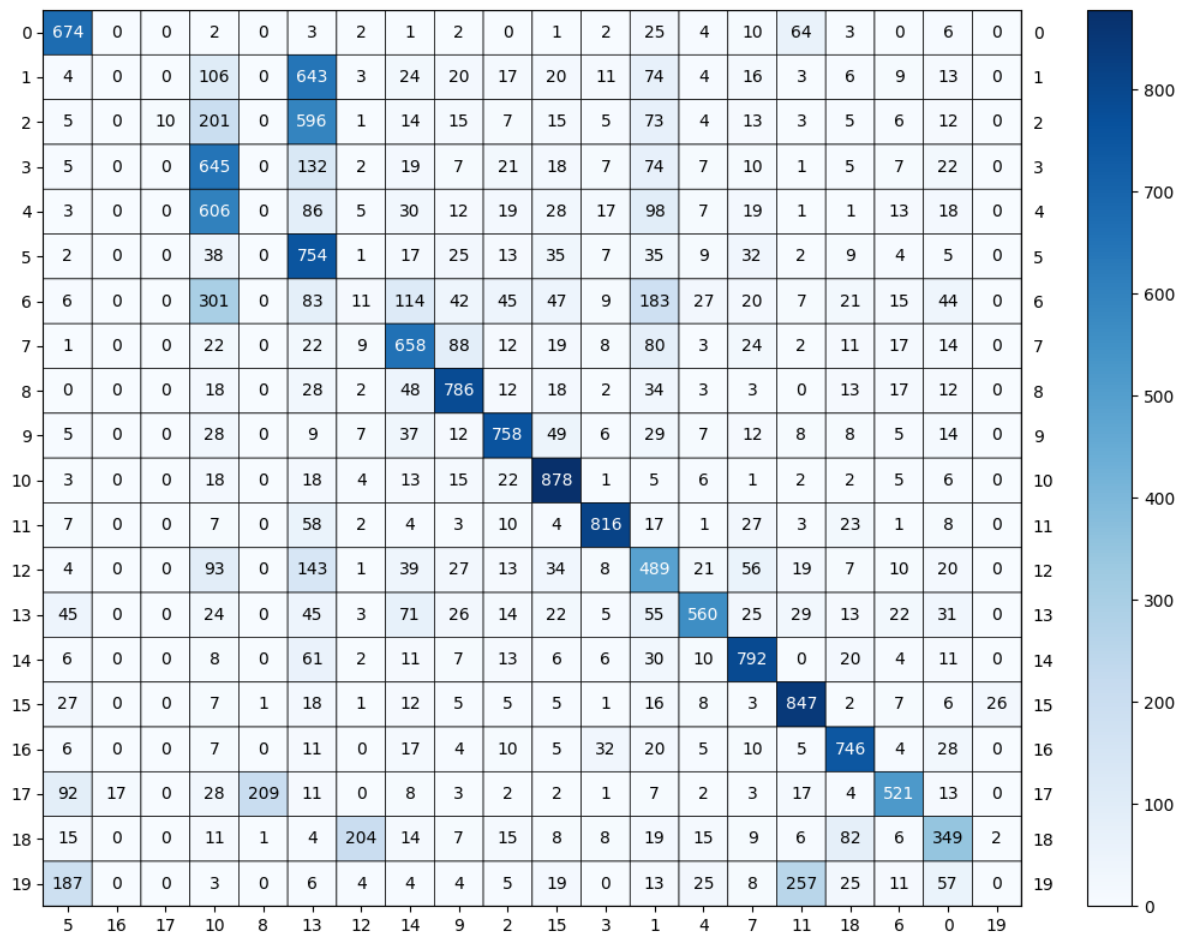
V-measure : 0.506

Adjusted Rand-Index : 0.393

Adjusted Mutual Information Score : 0.504

c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\utils\deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.

warnings.warn(



Number of Components: 20 Evaluating Metric: cosine

Homogeneity : 0.482

Completeness : 0.532

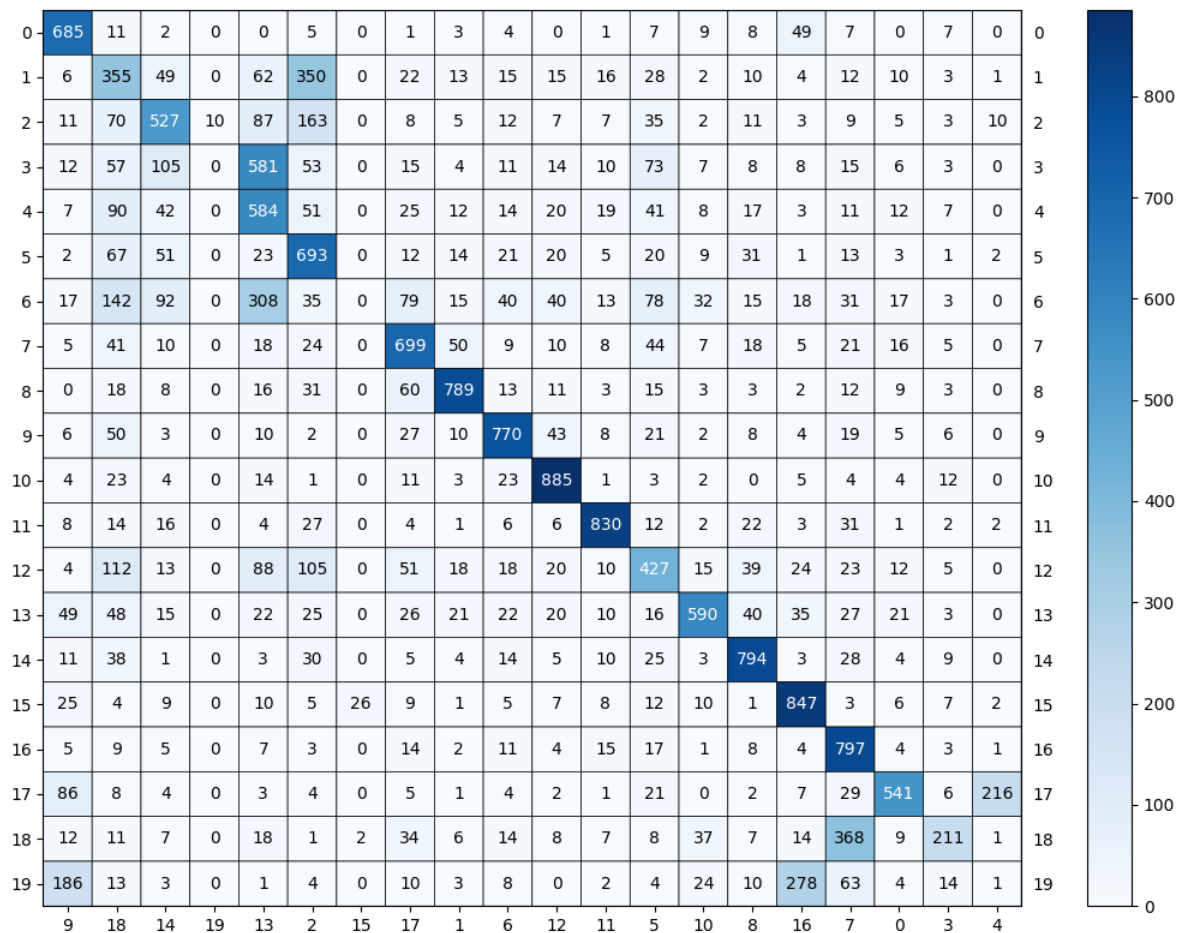
V-measure : 0.506

Adjusted Rand-Index : 0.368

Adjusted Mutual Information Score : 0.504

```
c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\utils
\deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_fi
nite' in 1.6 and will be removed in 1.8.
```

```
warnings.warn(
```



Number of Components: 20 Evaluating Metric: euclidean

Homogeneity : 0.499

Completeness : 0.529

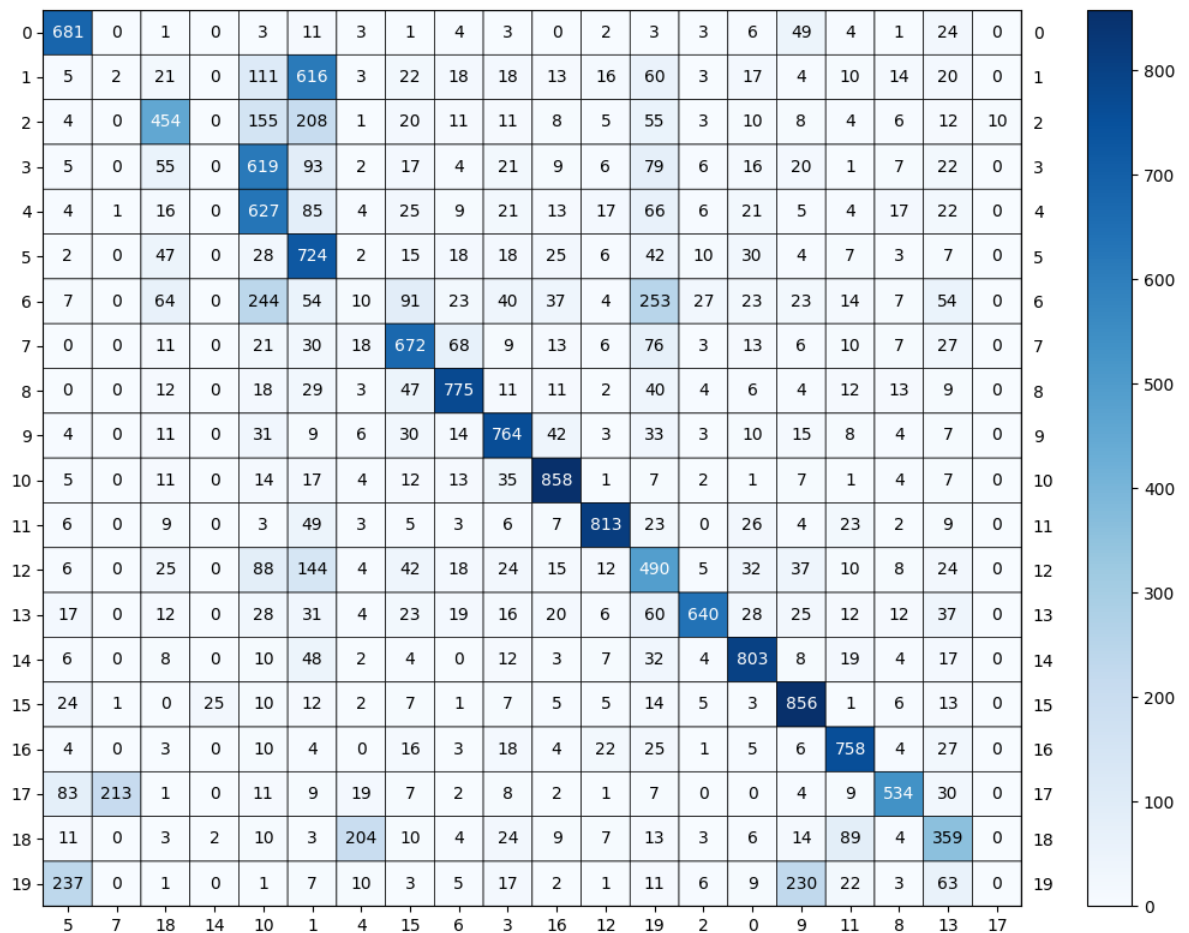
V-measure : 0.514

Adjusted Rand-Index : 0.403

Adjusted Mutual Information Score : 0.512

c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\utils\deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.

warnings.warn(



Number of Components: 200 Evaluating Metric: cosine

Homogeneity : 0.501

Completeness : 0.536

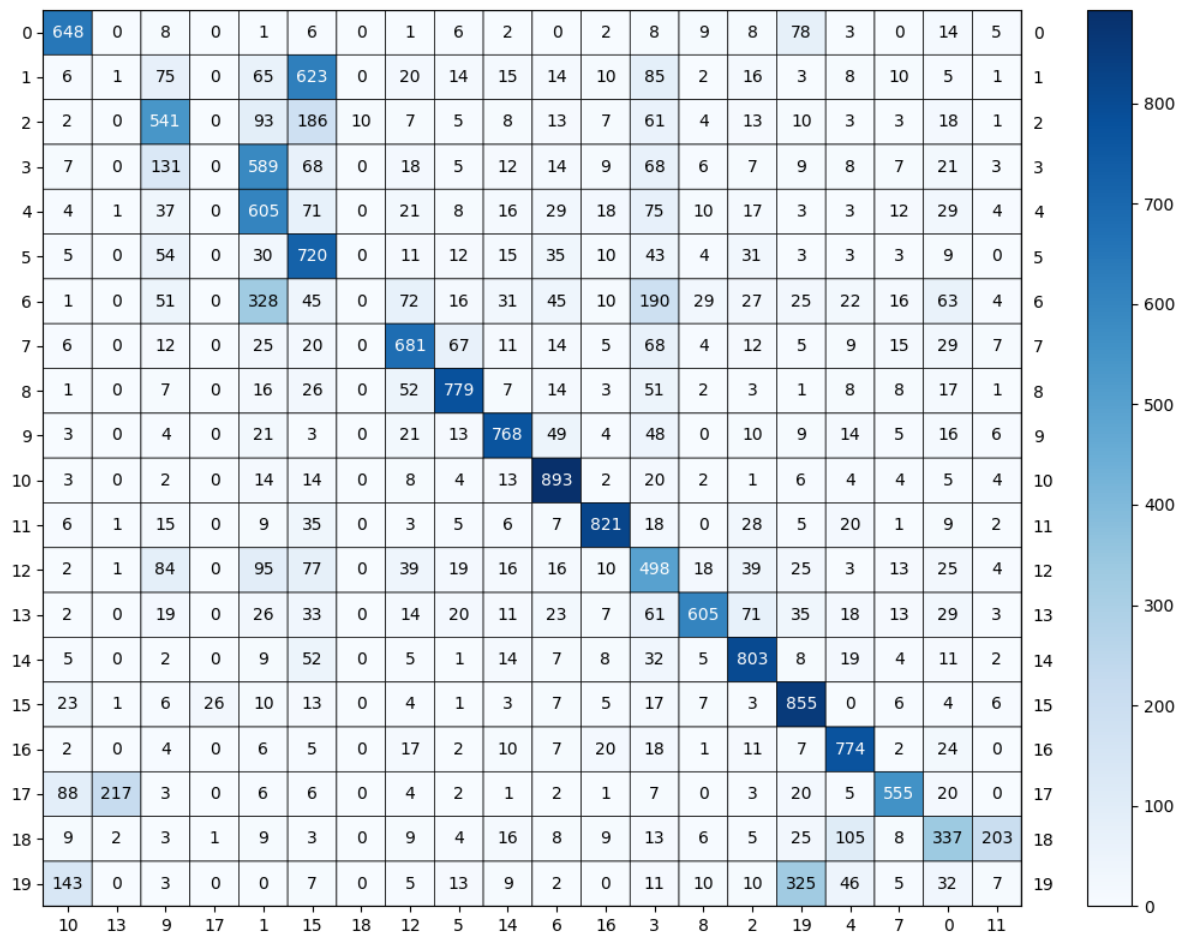
V-measure : 0.518

Adjusted Rand-Index : 0.396

Adjusted Mutual Information Score : 0.516

c:\Users\astro\repositories\UCLA\ECE 219\repo\.venv\Lib\site-packages\sklearn\utils
 \deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_fi
 nite' in 1.6 and will be removed in 1.8.

warnings.warn(



Number of Components: 200 Evaluating Metric: euclidean
 Homogeneity : 0.508
 Completeness : 0.543
 V-measure : 0.525
 Adjusted Rand-Index : 0.405
 Adjusted Mutual Information Score : 0.523

QUESTION 12: Analyze the contingency matrices. Which setting works best and why? What about for each metric choice?

Based on our evaluations of the 6 combinations, the iteration with a number of components of 200 and the metric of euclidean, was able to achieve the highest scores on all 5 metrics when compared against its counter parts. Indicating that these setting lead to the most optimal combination of hyper parameters for the full newsgroup data.

QUESTION 13: So far, we have attempted K-Means clustering with 4 different representation learning techniques (sparse TF-IDF representation, PCA-reduced, NMF-reduced, UMAP-reduced). Compare and contrast the clustering results across the 4 choices, and suggest an approach that is best for the K-Means clustering task on the 20-class text data. Choose any choice of clustering metrics for your comparison.

Based on the Contingency matrices created throughout the project document, we can see that for a K-means implementation with 20 classes, our best choices would likely be UMAP-

reduction, with 20 components, and a metric of euclidean, as this combination of hyperparameters will allow us to best represent the our dataset without lossing valuable information, or overfitting to our dataset.

QUESTION 14: Use UMAP to reduce the dimensionality properly, and perform Agglomerative clustering with `n_clusters=20` . Compare the performance of "ward" and "single" linkage criteria.

Report the five clustering evaluation metrics for each case.

```
In [78]: umap_full = umap.UMAP(n_components=20, metric='euclidean').fit_transform(tfidf_pred)
agg_cluster = AgglomerativeClustering(n_clusters = 20, linkage = 'ward')
agg_pred = agg_cluster.fit(umap_full)
y_pred = agg_pred.labels_

print("Homogeneity : ", homogeneity_score(data_full.target, y_pred))
print("Completeness : ", completeness_score(data_full.target, y_pred))
print("V-measure : ", v_measure_score(data_full.target, y_pred))
print("Adjusted Rand-Index : ", adjusted_rand_score(data_full.target, y_pred))
print("Adjusted Mutual Information Score : ", adjusted_mutual_info_score(data_full.

umap_full = umap.UMAP(n_components=20, metric='euclidean').fit_transform(tfidf_pred)
agg_cluster = AgglomerativeClustering(n_clusters = 20, linkage = 'single')
y_pred = agg_cluster.fit_predict(umap_full)
# y_pred = agg_pred.labels_

print("Homogeneity : " , homogeneity_score(data_full.target, y_pred))
print("Completeness : " , completeness_score(data_full.target, y_pred))
print("V-measure : " , v_measure_score(data_full.target, y_pred))
print("Adjusted Rand-Index : ", adjusted_rand_score(data_full.target, y_pred))
print("Adjusted Mutual Information Score : ", adjusted_mutual_info_score(data_full.

Homogeneity : 0.47905936599646803
Completeness : 0.5289501549781318
V-measure : 0.5027701040810756
Adjusted Rand-Index : 0.35129580544475375
Adjusted Mutual Information Score : 0.5010551171337507
Homogeneity : 0.020057199898966618
Completeness : 0.3569396172033896
V-measure : 0.03798021059772106
Adjusted Rand-Index : 0.0005978820980958837
Adjusted Mutual Information Score : 0.03252735577005047
```

QUESTION 15: Apply HDBSCAN on UMAP-transformed 20-category data. Use `min_cluster_size=100`.

Vary the min cluster size among 20, 100, 200 and report your findings in terms of the five clustering evaluation metrics - you will plot the best contingency matrix in the next question. Feel free to try modifying other parameters in HDBSCAN to get better performance.

```
In [77]: cluster_sizes = [20, 100, 200]
```

```

umap_data = umap.UMAP(n_components=20, metric='euclidean').fit_transform(tfidf_pred

for i in cluster_sizes:
    scan = hdbscan.HDBSCAN(min_cluster_size = i)
    y_pred = scan.fit_predict(umap_data)
    print("Cluser Size : ", i)
    print("Homogeneity : " , homogeneity_score(data_full.target, y_pred))
    print("Completeness : " , completeness_score(data_full.target, y_pred))
    print("V-measure : " , v_measure_score(data_full.target, y_pred))
    print("Adjusted Rand-Index : ", adjusted_rand_score(data_full.target, y_pred))
    print("Adjusted Mutual Information Score : ", adjusted_mutual_info_score(data_f

```

```

Cluser Size : 20
Homogeneity : 0.38487866199768805
Completeness : 0.3513118772083331
V-measure : 0.36733002678807974
Adjusted Rand-Index : 0.03203092211313788
Adjusted Mutual Information Score : 0.3453488956464685
Cluser Size : 100
Homogeneity : 0.013375875417440685
Completeness : 0.45514710358558597
V-measure : 0.025988014364309076
Adjusted Rand-Index : 0.000578056376781946
Adjusted Mutual Information Score : 0.025320405662514343
Cluser Size : 200
Homogeneity : 0.013375875417440685
Completeness : 0.45514710358558597
V-measure : 0.025988014364309076
Adjusted Rand-Index : 0.000578056376781946
Adjusted Mutual Information Score : 0.025320405662514343

```

QUESTION 16: Contingency matrix Plot the contingency matrix for the best clustering model from Question 15. How many clusters are given by the model? What does "-1" mean for the clustering labels?

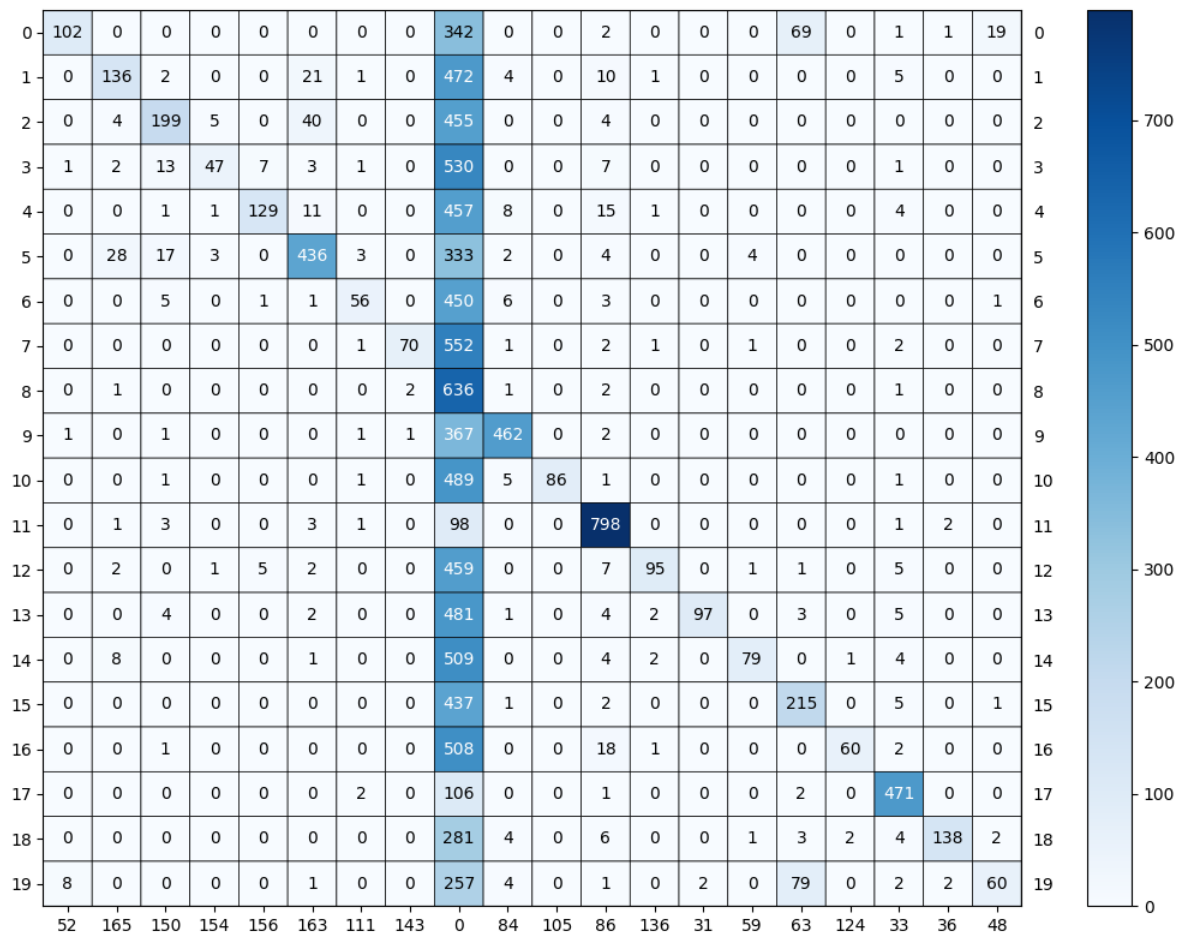
Interpret the contingency matrix considering the answer to these questions.

```

In [79]: umap_data = umap.UMAP(n_components=20, metric='euclidean').fit_transform(tfidf_pred
scan = hdbscan.HDBSCAN(min_cluster_size = 20)
y_pred = scan.fit_predict(umap_data)

cm = contingency_matrix(data_full.target, y_pred)
rows, cols = linear_sum_assignment(cm, maximize=True)
reordered_mat = cm[np.ix_(rows, cols)]
plotmat.plot_mat(reordered_mat, xticklabels=cols, yticklabels=rows, size=(10,8))

```



Based on the resulting contingency matrix created using HDBSCAN, we can see that the number of clusters is 20, which was the best from our results in question 15. According to HDBSCAN documentation, -1 signifies a cluster that has been classified by the model as noise and do not belong to any of the identified clusters.

QUESTION 17: Based on your experiments, which dimensionality reduction technique and clustering methods worked best together for 20-class text data and why? Follow the table below. If UMAP takes too long to converge, consider running it once and saving the intermediate results in a pickle file.

Hint: DBSCAN and HDBSCAN do not accept the number of clusters as an input parameter. So pay close attention to how the different clustering metrics are being computed for these methods.

```
In [82]: def compare_models(models_metrics):
    best_model = None
    best_score = -float('inf')
    for model, metrics in models_metrics.items():
        cumulative_score = np.sum(metrics)
        if cumulative_score > best_score:
            best_score = cumulative_score
            best_model = model
```

```
return best_model, best_score
```

```
In [ ]: # Takes 1 hour and 12 minutes to fully run
dimensionality_reduction = [TruncatedSVD(n_components=5, random_state=42),
                             TruncatedSVD(n_components=20, random_state=42),
                             TruncatedSVD(n_components=200, random_state=42),
                             NMF(n_components=5, random_state=42),
                             NMF(n_components=20, random_state=42),
                             NMF(n_components=200, random_state=42),
                             umap.UMAP(n_components=5, metric='euclidean'),
                             umap.UMAP(n_components=20, metric='euclidean'),
                             umap.UMAP(n_components=200, metric='euclidean')]

cluster_techniques = [KMeans(n_clusters=10, random_state=42, max_iter=1000, n_init=
                           KMeans(n_clusters=20, random_state=42, max_iter=1000, n_init=
                           KMeans(n_clusters=50, random_state=42, max_iter=1000, n_init=
                           AgglomerativeClustering(n_clusters=20),
                           hdbscan.HDBSCAN(min_cluster_size=100),
                           hdbscan.HDBSCAN(min_cluster_size=200)]

model_metrics = {}
for i in dimensionality_reduction:
    for c in cluster_techniques:
        if i is None:
            y_pred = c.fit_predict(tfidf_preds)
            hm_score = homogeneity_score(data_full.target, y_pred)
            comp_score = completeness_score(data_full.target, y_pred)
            v_score = v_measure_score(data_full.target, y_pred)
            rand_score = adjusted_rand_score(data_full.target, y_pred)
            mut_score = adjusted_mutual_info_score(data_full.target, y_pred)
            metrics_array = [hm_score, comp_score, v_score, rand_score, mut_score]
            model_metrics[f"{i},{c}"] = metrics_array
            print("Dimensionality Reduction Type:", i, ", Clustering Type: ", c)
            print("Homogeneity : ", hm_score)
            print("Completeness : ", comp_score)
            print("V-measure : ", v_score)
            print("Adjusted Rand-Index : ", rand_score)
            print("Adjusted Mutual Information Score : ", mut_score)
        else:
            method = i.fit_transform(tfidf_preds)
            y_pred = c.fit_predict(method)
            hm_score = homogeneity_score(data_full.target, y_pred)
            comp_score = completeness_score(data_full.target, y_pred)
            v_score = v_measure_score(data_full.target, y_pred)
            rand_score = adjusted_rand_score(data_full.target, y_pred)
            mut_score = adjusted_mutual_info_score(data_full.target, y_pred)
            metrics_array = [hm_score, comp_score, v_score, rand_score, mut_score]
            model_metrics[f"{i},{c}"] = metrics_array
            print("Dimensionality Reduction Type:", i, ", Clustering Type: ", c)
            print("Homogeneity : ", hm_score)
            print("Completeness : ", comp_score)
            print("V-measure : ", v_score)
            print("Adjusted Rand-Index : ", rand_score)
            print("Adjusted Mutual Information Score : ", mut_score)

best_model, best_score = compare_models(model_metrics)
```

```
print("Best Model Found is ", best_model)
print("With a score of ", best_score)
```

Dimentionallity Reduction Type: TruncatedSVD(n_components=5, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.25273865828769104
Completeness : 0.3639992217811067
V-measure : 0.2983331425028029
Adjusted Rand-Index : 0.11262133015505373
Adjusted Mutual Information Score : 0.2970693666782106
Dimentionallity Reduction Type: TruncatedSVD(n_components=5, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.3126207568897337
Completeness : 0.33144480930213405
V-measure : 0.32175769856430775
Adjusted Rand-Index : 0.12248763485114002
Adjusted Mutual Information Score : 0.3194995928145638
Dimentionallity Reduction Type: TruncatedSVD(n_components=5, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.3617757672962524
Completeness : 0.29042847974142433
V-measure : 0.3221996378599729
Adjusted Rand-Index : 0.09263520495490955
Adjusted Mutual Information Score : 0.31708461648668484
Dimentionallity Reduction Type: TruncatedSVD(n_components=5, random_state=42) , Clustering Type: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.30310687128330555
Completeness : 0.32738252149182245
V-measure : 0.31477735530316253
Adjusted Rand-Index : 0.12005558568374378
Adjusted Mutual Information Score : 0.3124748822850994
Dimentionallity Reduction Type: TruncatedSVD(n_components=5, random_state=42) , Clustering Type: HDBSCAN(min_cluster_size=100)
Homogeneity : 0.0
Completeness : 1.0
V-measure : 0.0
Adjusted Rand-Index : 0.0
Adjusted Mutual Information Score : 0.0
Dimentionallity Reduction Type: TruncatedSVD(n_components=5, random_state=42) , Clustering Type: HDBSCAN(min_cluster_size=200)
Homogeneity : 0.0
Completeness : 1.0
V-measure : 0.0
Adjusted Rand-Index : 0.0
Adjusted Mutual Information Score : 0.0
Dimentionallity Reduction Type: TruncatedSVD(n_components=20, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.2761589150533012
Completeness : 0.43903515927729825
V-measure : 0.3390505531516107
Adjusted Rand-Index : 0.10122841884333234
Adjusted Mutual Information Score : 0.33780371066329795
Dimentionallity Reduction Type: TruncatedSVD(n_components=20, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.2880305059891321
Completeness : 0.38347176056805204
V-measure : 0.32896855522245194
Adjusted Rand-Index : 0.09297012622849953
Adjusted Mutual Information Score : 0.3264581794062097

Dimentionallity Reduction Type: TruncatedSVD(n_components=20, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.39611636948769197
Completeness : 0.3383394677762406
V-measure : 0.3649553719368441
Adjusted Rand-Index : 0.1384609662307626
Adjusted Mutual Information Score : 0.3599411973709435
Dimentionallity Reduction Type: TruncatedSVD(n_components=20, random_state=42) , Clustering Type: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.33040464820347853
Completeness : 0.4333175252151034
V-measure : 0.37492724307908576
Adjusted Rand-Index : 0.12413059838732861
Adjusted Mutual Information Score : 0.37260624076173743
Dimentionallity Reduction Type: TruncatedSVD(n_components=20, random_state=42) , Clustering Type: HDBSCAN(min_cluster_size=100)
Homogeneity : 0.0
Completeness : 1.0
V-measure : 0.0
Adjusted Rand-Index : 0.0
Adjusted Mutual Information Score : 0.0
Dimentionallity Reduction Type: TruncatedSVD(n_components=20, random_state=42) , Clustering Type: HDBSCAN(min_cluster_size=200)
Homogeneity : 0.0
Completeness : 1.0
V-measure : 0.0
Adjusted Rand-Index : 0.0
Adjusted Mutual Information Score : 0.0
Dimentionallity Reduction Type: TruncatedSVD(n_components=200, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.2609774347277058
Completeness : 0.5093927418070324
V-measure : 0.3451328077722315
Adjusted Rand-Index : 0.0771553849700257
Adjusted Mutual Information Score : 0.34380329298014695
Dimentionallity Reduction Type: TruncatedSVD(n_components=200, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.29983314914549714
Completeness : 0.4371374610581335
V-measure : 0.35569478550118055
Adjusted Rand-Index : 0.0950019331687476
Adjusted Mutual Information Score : 0.3531683559256488
Dimentionallity Reduction Type: TruncatedSVD(n_components=200, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.33974086615180077
Completeness : 0.33392134864845885
V-measure : 0.3368059710757022
Adjusted Rand-Index : 0.09536142900515182
Adjusted Mutual Information Score : 0.3310842014816762
Dimentionallity Reduction Type: TruncatedSVD(n_components=200, random_state=42) , Clustering Type: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.22503950308985432
Completeness : 0.40902480948326814
V-measure : 0.2903388128059071
Adjusted Rand-Index : 0.04863740427366599
Adjusted Mutual Information Score : 0.2872824945873852

Dimentionallity Reduction Type: TruncatedSVD(n_components=200, random_state=42) , Cl
 ustering Type: HDBSCAN(min_cluster_size=100)
 Homogeneity : 0.0
 Completeness : 1.0
 V-measure : 0.0
 Adjusted Rand-Index : 0.0
 Adjusted Mutual Information Score : 0.0
 Dimentionallity Reduction Type: TruncatedSVD(n_components=200, random_state=42) , Cl
 ustering Type: HDBSCAN(min_cluster_size=200)
 Homogeneity : 0.0
 Completeness : 1.0
 V-measure : 0.0
 Adjusted Rand-Index : 0.0
 Adjusted Mutual Information Score : 0.0
 Dimentionallity Reduction Type: NMF(n_components=5, random_state=42) , Clustering Ty
 pe: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
 Homogeneity : 0.25649428568550203
 Completeness : 0.3676581095715578
 V-measure : 0.30217685586931053
 Adjusted Rand-Index : 0.10408478491728526
 Adjusted Mutual Information Score : 0.3009228753354001
 Dimentionallity Reduction Type: NMF(n_components=5, random_state=42) , Clustering Ty
 pe: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
 Homogeneity : 0.2931683187383885
 Completeness : 0.3101435167296279
 V-measure : 0.30141710479358386
 Adjusted Rand-Index : 0.10550265570268441
 Adjusted Mutual Information Score : 0.2990926697960212
 Dimentionallity Reduction Type: NMF(n_components=5, random_state=42) , Clustering Ty
 pe: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
 Homogeneity : 0.3299195953342298
 Completeness : 0.2627904885764448
 V-measure : 0.29255359070925285
 Adjusted Rand-Index : 0.07647432708367888
 Adjusted Mutual Information Score : 0.28723456699452027
 Dimentionallity Reduction Type: NMF(n_components=5, random_state=42) , Clustering Ty
 pe: AgglomerativeClustering(n_clusters=20)
 Homogeneity : 0.2872534363748666
 Completeness : 0.30222936082300966
 V-measure : 0.2945511654707259
 Adjusted Rand-Index : 0.11037234598060279
 Adjusted Mutual Information Score : 0.2922115256442744
 Dimentionallity Reduction Type: NMF(n_components=5, random_state=42) , Clustering Ty
 pe: HDBSCAN(min_cluster_size=100)
 Homogeneity : 0.018042395354984052
 Completeness : 0.14975151256474453
 V-measure : 0.03220469715733114
 Adjusted Rand-Index : 0.0008352298017490346
 Adjusted Mutual Information Score : 0.03161902552279655
 Dimentionallity Reduction Type: NMF(n_components=5, random_state=42) , Clustering Ty
 pe: HDBSCAN(min_cluster_size=200)
 Homogeneity : 0.011350452851269456
 Completeness : 0.12530464512554135
 V-measure : 0.02081538834041675
 Adjusted Rand-Index : 0.00022356061900455452
 Adjusted Mutual Information Score : 0.02020529992144077

Dimentionallity Reduction Type: NMF(n_components=20, random_state=42) , Clustering T
ype: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.16871361838082372
Completeness : 0.5362928185031597
V-measure : 0.25667822926901274
Adjusted Rand-Index : 0.025189383124498123
Adjusted Mutual Information Score : 0.2549250310861421
Dimentionallity Reduction Type: NMF(n_components=20, random_state=42) , Clustering T
ype: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.2746972019021395
Completeness : 0.3893870356552565
V-measure : 0.32213843696967087
Adjusted Rand-Index : 0.07128311820408853
Adjusted Mutual Information Score : 0.31953319755372406
Dimentionallity Reduction Type: NMF(n_components=20, random_state=42) , Clustering T
ype: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.37196007906519885
Completeness : 0.3242952887551911
V-measure : 0.3464961473071744
Adjusted Rand-Index : 0.12201005235975028
Adjusted Mutual Information Score : 0.3412428467873473
Dimentionallity Reduction Type: NMF(n_components=20, random_state=42) , Clustering T
ype: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.2768374244938308
Completeness : 0.3949356452900103
V-measure : 0.3255056560039718
Adjusted Rand-Index : 0.06410355909524232
Adjusted Mutual Information Score : 0.3229069646927765
Dimentionallity Reduction Type: NMF(n_components=20, random_state=42) , Clustering T
ype: HDBSCAN(min_cluster_size=100)
Homogeneity : 0.006673916486519139
Completeness : 0.12738529922358485
V-measure : 0.012683333169229937
Adjusted Rand-Index : 3.1645109232318096e-05
Adjusted Mutual Information Score : 0.012038709480669832
Dimentionallity Reduction Type: NMF(n_components=20, random_state=42) , Clustering T
ype: HDBSCAN(min_cluster_size=200)
Homogeneity : 0.0
Completeness : 1.0
V-measure : 0.0
Adjusted Rand-Index : 0.0
Adjusted Mutual Information Score : 0.0
Dimentionallity Reduction Type: NMF(n_components=200, random_state=42) , Clustering
Type: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.0692691383830504
Completeness : 0.40107876168497913
V-measure : 0.1181354492776133
Adjusted Rand-Index : 0.00498138969246422
Adjusted Mutual Information Score : 0.11572861944688817
Dimentionallity Reduction Type: NMF(n_components=200, random_state=42) , Clustering
Type: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.0968298545793245
Completeness : 0.3354783956939782
V-measure : 0.1502831567476059
Adjusted Rand-Index : 0.005886402472406172
Adjusted Mutual Information Score : 0.14580247457679077

Dimentionallity Reduction Type: NMF(n_components=200, random_state=42) , Clustering Type: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.16176739651248698
Completeness : 0.257727141573628
V-measure : 0.19877183094294462
Adjusted Rand-Index : 0.009077259642256795
Adjusted Mutual Information Score : 0.1899536138916994
Dimentionallity Reduction Type: NMF(n_components=200, random_state=42) , Clustering Type: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.1010158535752369
Completeness : 0.35472971837494793
V-measure : 0.1572514468404556
Adjusted Rand-Index : 0.007399304083035572
Adjusted Mutual Information Score : 0.15271962544680398
Dimentionallity Reduction Type: NMF(n_components=200, random_state=42) , Clustering Type: HDBSCAN(min_cluster_size=100)
Homogeneity : 0.0
Completeness : 1.0
V-measure : 0.0
Adjusted Rand-Index : 0.0
Adjusted Mutual Information Score : 0.0
Dimentionallity Reduction Type: NMF(n_components=200, random_state=42) , Clustering Type: HDBSCAN(min_cluster_size=200)
Homogeneity : 0.0
Completeness : 1.0
V-measure : 0.0
Adjusted Rand-Index : 0.0
Adjusted Mutual Information Score : 0.0
Dimentionallity Reduction Type: UMAP(n_components=5, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.3986837912544073
Completeness : 0.5843308322641636
V-measure : 0.4739771433309075
Adjusted Rand-Index : 0.2511056109711637
Adjusted Mutual Information Score : 0.4730223546143701
Dimentionallity Reduction Type: UMAP(n_components=5, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.5074283036081111
Completeness : 0.5409278040799056
V-measure : 0.5236428270619873
Adjusted Rand-Index : 0.40588207428305056
Adjusted Mutual Information Score : 0.5220390593957115
Dimentionallity Reduction Type: UMAP(n_components=5, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.5699559580140793
Completeness : 0.4578535804893018
V-measure : 0.5077913102031363
Adjusted Rand-Index : 0.3677076016137933
Adjusted Mutual Information Score : 0.5040620995506321
Dimentionallity Reduction Type: UMAP(n_components=5, tqdm_kwds={'bar_format': '{desc}

c}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]", 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.4887317332825636
Completeness : 0.5146456621942461
V-measure : 0.5013540620795457
Adjusted Rand-Index : 0.37706892236663747
Adjusted Mutual Information Score : 0.49968597959687555
Dimentionallity Reduction Type: UMAP(n_components=5, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]", 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: HDBSCAN(min_cluster_size=100)
Homogeneity : 0.2717337961244501
Completeness : 0.5928219231236855
V-measure : 0.37265325533046806
Adjusted Rand-Index : 0.08277277156280796
Adjusted Mutual Information Score : 0.371482047092798
Dimentionallity Reduction Type: UMAP(n_components=5, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]", 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: HDBSCAN(min_cluster_size=200)
Homogeneity : 0.013906166787540425
Completeness : 0.49490187459805834
V-measure : 0.027052198282422068
Adjusted Rand-Index : 0.0005964497395942732
Adjusted Mutual Information Score : 0.026369565026625134
Dimentionallity Reduction Type: UMAP(n_components=20, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]", 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.39445037877495115
Completeness : 0.5780252323140412
V-measure : 0.46891103330073736
Adjusted Rand-Index : 0.2493576883355176
Adjusted Mutual Information Score : 0.4679471173468865
Dimentionallity Reduction Type: UMAP(n_components=20, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]", 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.5119337513721676
Completeness : 0.5381320725495577
V-measure : 0.5247060981474436
Adjusted Rand-Index : 0.41551890463133057
Adjusted Mutual Information Score : 0.5231175542682646
Dimentionallity Reduction Type: UMAP(n_components=20, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]", 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.5750284037922084
Completeness : 0.47269349466591215
V-measure : 0.5188632329260514
Adjusted Rand-Index : 0.41294533327743965
Adjusted Mutual Information Score : 0.515143073844444
Dimentionallity Reduction Type: UMAP(n_components=20, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]", 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.4911007246827806
Completeness : 0.5289822658448694
V-measure : 0.5093381156495482

Adjusted Rand-Index : 0.37191898467480083
Adjusted Mutual Information Score : 0.5076773718257355
Dimentionallity Reduction Type: UMAP(n_components=20, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: HDBSCAN(min_cluster_size=100)
Homogeneity : 0.014010022581543269
Completeness : 0.4885939357545715
V-measure : 0.02723898990285681
Adjusted Rand-Index : 0.0006287039623015205
Adjusted Mutual Information Score : 0.026566033073556997
Dimentionallity Reduction Type: UMAP(n_components=20, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: HDBSCAN(min_cluster_size=200)
Homogeneity : 0.013557469049912231
Completeness : 0.48286758874085706
V-measure : 0.026374423638861062
Adjusted Rand-Index : 0.0005986558796979328
Adjusted Mutual Information Score : 0.025699141711480303
Dimentionallity Reduction Type: UMAP(n_components=200, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: KMeans(max_iter=1000, n_clusters=10, n_init=30, random_state=42)
Homogeneity : 0.395120049919619
Completeness : 0.5833859311223732
V-measure : 0.47114168475907026
Adjusted Rand-Index : 0.24281841749217073
Adjusted Mutual Information Score : 0.4701790519945202
Dimentionallity Reduction Type: UMAP(n_components=200, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
Homogeneity : 0.5073754024789282
Completeness : 0.5431840793698351
V-measure : 0.5246694654650517
Adjusted Rand-Index : 0.4014722822240333
Adjusted Mutual Information Score : 0.5230658817578807
Dimentionallity Reduction Type: UMAP(n_components=200, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: KMeans(max_iter=1000, n_clusters=50, n_init=30, random_state=42)
Homogeneity : 0.566504621931102
Completeness : 0.4597641592492361
V-measure : 0.5075834440045996
Adjusted Rand-Index : 0.404035056610579
Adjusted Mutual Information Score : 0.5038233860995058
Dimentionallity Reduction Type: UMAP(n_components=200, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}) , Clustering Type: AgglomerativeClustering(n_clusters=20)
Homogeneity : 0.48458902760676054
Completeness : 0.5390625372998407
V-measure : 0.5103763814266179
Adjusted Rand-Index : 0.3539967064684246
Adjusted Mutual Information Score : 0.5086687465481179
Dimentionallity Reduction Type: UMAP(n_components=200, tqdm_kwds={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs com

```

pleted', 'disable': True)) , Clustering Type: HDBSCAN(min_cluster_size=100)
Homogeneity : 0.01394077175109458
Completeness : 0.4825762697636899
V-measure : 0.02709870988011164
Adjusted Rand-Index : 0.0006384068573917776
Adjusted Mutual Information Score : 0.026425332954461
Dimensionality Reduction Type: UMAP(n_components=200, tqdm_kws={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True)) , Clustering Type: HDBSCAN(min_cluster_size=200)
Homogeneity : 0.013754781970847894
Completeness : 0.45727709920099674
V-measure : 0.026706246652025794
Adjusted Rand-Index : 0.0006068234788160461
Adjusted Mutual Information Score : 0.026039459057344345
Best Model Found is UMAP(n_components=20, tqdm_kws={'bar_format': '{desc}: {percentage:3.0f}%| {bar} {n_fmt}/{total_fmt} [{elapsed}]', 'desc': 'Epochs completed', 'disable': True}), KMeans(max_iter=1000, n_clusters=20, n_init=30, random_state=42)
With a score of 2.513408380968764

```

We can safely assume that without dimensionality reduction, we will have a suboptimal clustering result, furthermore, from our experiments, we can see that of the available techniques, a UMAP with 20 components and Kmeans and 20 clusters performs best with the following metrics:

- Homogeneity : 0.5119337513721676
- Completeness : 0.5381320725495577
- V-measure : 0.5247060981474436
- Adjusted Rand-Index : 0.41551890463133057
- Adjusted Mutual Information Score : 0.5231175542682646

QUESTION 18: Extra credit: If you can find creative ways to further enhance the clustering performance, report your method and the results you obtain.

```

In [103... from sklearn.cluster import SpectralClustering
spectral = SpectralClustering(n_clusters=20, affinity='nearest_neighbors')
y_pred = spectral.fit_predict(tfidf_preds)

cm = contingency_matrix(data_full.target, y_pred)

hm_score = homogeneity_score(data_full.target, y_pred)
comp_score = completeness_score(data_full.target, y_pred)
v_score = v_measure_score(data_full.target, y_pred)
rand_score = adjusted_rand_score(data_full.target, y_pred)
mut_score = adjusted_mutual_info_score(data_full.target, y_pred)

print("Homogeneity : " , hm_score)
print("Completeness : " , comp_score)
print("V-measure : " , v_score)
print("Adjusted Rand-Index : ", rand_score)
print("Adjusted Mutual Information Score : ", mut_score)

rows, cols = linear_sum_assignment(cm, maximize=True)
reordered_mat = cm[np.ix_(rows, cols)]

```

```
plotmat.plot_mat(reordered_mat, xticklabels=cols, yticklabels=rows, size=(10,8))
```

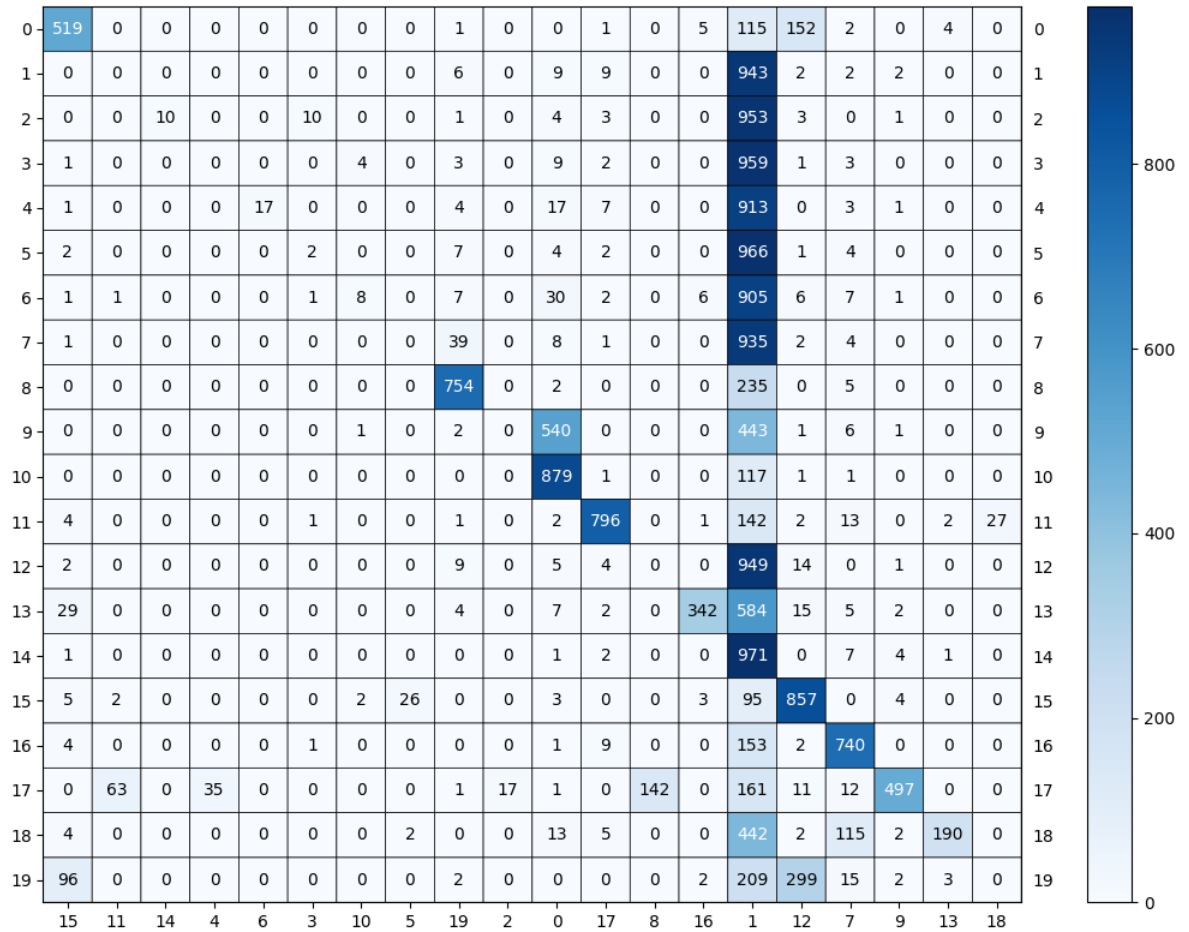
Homogeneity : 0.35324846286571226

Completeness : 0.6647802762512584

V-measure : 0.46134770405969555

Adjusted Rand-Index : 0.09668461226948066

Adjusted Mutual Information Score : 0.4589353981944944



Unfortunately, we were not able to get better results than our UMAP and KMeans implementation from part 17, however, some possible suggestions would be playing with UMAP hyperparameters like the number of neighbors, minimum distance from the center of the cluster, or attempting different embedding strategies to find the most optimal configuration.

Part 2 - Deep Learning and Clustering of Image Data

In this part, we aim to cluster the images of the `tf flowers` dataset. This dataset consists of images of five types of flowers. Explore this [link](#) to see actual samples of the data. Extracting meaningful features from images has a long history in computer vision. Instead of considering the raw pixel values as features, researchers have explored various hand-engineered feature extraction methods, e.g. [5]. With the recent rise of “deep learning”, these methods are replaced with using appropriate neural networks. Particularly, one can adopt a neural network already trained to classify another large dataset of images³. These pre-trained networks have been trained to morph the highly non-smooth scatter of images in the higher dimension, into smooth lower-dimensional manifolds.

```
In [2]: # Imports
import torch
import torch.nn as nn
from torchvision import transforms, datasets
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from tqdm import tqdm
import requests
import os
import tarfile

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import confusion_matrix, adjusted_rand_score, adjusted_mutual_
from sklearn.pipeline import Pipeline
from sklearn.base import TransformerMixin
from sklearn.cluster import AgglomerativeClustering

from sklearn.decomposition import TruncatedSVD
!pip install umap-learn
from umap import UMAP
!pip install hdbscan
import hdbscan
```

```

Collecting umap-learn
  Downloading umap_learn-0.5.7-py3-none-any.whl.metadata (21 kB)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (1.26.4)
Requirement already satisfied: scipy>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (1.13.1)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (1.6.1)
Requirement already satisfied: numba>=0.51.2 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (0.61.0)
Collecting pynndescent>=0.5 (from umap-learn)
  Downloading pynndescent-0.5.13-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from umap-learn) (4.67.1)
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba>=0.51.2->umap-learn) (0.44.0)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.11/dist-packages (from pynndescent>=0.5->umap-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.22->umap-learn) (3.5.0)
Downloading umap_learn-0.5.7-py3-none-any.whl (88 kB)
  _____ 0.0/88.8 kB ? eta -:--:--
  _____ 88.8/88.8 kB 8.8 MB/s eta 0:00:00
Downloading pynndescent-0.5.13-py3-none-any.whl (56 kB)
  _____ 0.0/56.9 kB ? eta -:--:--
  _____ 56.9/56.9 kB 5.4 MB/s eta 0:00:00
Installing collected packages: pynndescent, umap-learn
Successfully installed pynndescent-0.5.13 umap-learn-0.5.7
Collecting hdbscan
  Downloading hdbscan-0.8.40-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (15 kB)
Requirement already satisfied: numpy<3,>=1.20 in /usr/local/lib/python3.11/dist-packages (from hdbscan) (1.26.4)
Requirement already satisfied: scipy>=1.0 in /usr/local/lib/python3.11/dist-packages (from hdbscan) (1.13.1)
Requirement already satisfied: scikit-learn>=0.20 in /usr/local/lib/python3.11/dist-packages (from hdbscan) (1.6.1)
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.11/dist-packages (from hdbscan) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.20->hdbscan) (3.5.0)
Downloading hdbscan-0.8.40-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (4.6 MB)
  _____ 4.6/4.6 MB 54.5 MB/s eta 0:00:00
Installing collected packages: hdbscan
Successfully installed hdbscan-0.8.40

```

In this project, we use a VGG network [6] pre-trained on the ImageNet dataset [7]. We provide a helper codebase (check Week 4 in BruinLearn), which guides you through the necessary steps for loading the VGG network and for using it for feature extraction.

QUESTION 19:

In a brief paragraph discuss: If the VGG network is trained on a dataset with perhaps totally

different classes as targets, why would one expect the features derived from such a network to have discriminative power for a custom dataset?

ANSWER:

Even if a VGG network was originally trained on a different dataset with unrelated classes as targets, it can still perform quite well on a completely different dataset. This is possible because of the concept of transfer learning: a ML technique where a model trained for one task, is reused as a starting point for another (typically related) task, effectively reducing the amount of data required and making for a more efficient training process. This is possible because of how a VGG network processes images: The early layers learn to detect basic visual elements like edges, corners, and basic textures. The middle layers combine these to identify more complex patterns like shapes, objects and more complex textures. Later layers identify higher-level features more specific to the classes it was trained on. The early and middle layers are the key to transfer learning as they are largely universal and generalizable across different datasets, regardless of the dataset the network was trained on. That being said, performance across separate datasets will ultimately depend on similarity between the two and the size/diversity of the data it was trained on.

Use the helper code to load the flowers dataset, and extract their features. To perform computations on deep neural networks fast enough, GPU resources are often required. GPU resources can be freely accessed through "Google Colab".

```
In [4]: filename = './flowers_features_and_labels.npz'

if os.path.exists(filename):
    file = np.load(filename)
    f_all, y_all = file['f_all'], file['y_all']
else:
    if not os.path.exists('./flower_photos'):
        # download the flowers dataset and extract its images
        url = 'http://download.tensorflow.org/example_images/flower_photos.tgz'
        with open('./flower_photos.tgz', 'wb') as file:
            file.write(requests.get(url).content)
        with tarfile.open('./flower_photos.tgz') as file:
            file.extractall('.')
        os.remove('./flower_photos.tgz')

    class FeatureExtractor(nn.Module):
        def __init__(self):
            super().__init__()

            vgg = torch.hub.load('pytorch/vision:v0.10.0', 'vgg16', pretrained=True)

            # Extract VGG-16 Feature Layers
            self.features = list(vgg.features)
            self.features = nn.Sequential(*self.features)
            # Extract VGG-16 Average Pooling Layer
```

```

self.pooling = vgg.avgpool
# Convert the image into one-dimensional vector
self.flatten = nn.Flatten()
# Extract the first part of fully-connected Layer from VGG16
self.fc = vgg.classifier[0]

def forward(self, x):
    # It will take the input 'x' until it returns the feature vector called
    out = self.features(x)
    out = self.pooling(out)
    out = self.flatten(out)
    out = self.fc(out)
    return out

# Initialize the model
assert torch.cuda.is_available()
# device = torch.device("cpu") # Force the model to run on CPU (I ran out of fr
feature_extractor = FeatureExtractor().cuda().eval()
# feature_extractor = FeatureExtractor().to(device).eval()

dataset = datasets.ImageFolder(root='./flower_photos',
                                transform=transforms.Compose([transforms.Resize(
                                                                    transforms.CenterC
                                                                    transforms.ToTens
                                                                    transforms.Normali

dataloader = DataLoader(dataset, batch_size=64, shuffle=True)

# Extract features and store them on disk
f_all, y_all = np.zeros((0, 4096)), np.zeros((0,))
for x, y in tqdm(dataloader):
    with torch.no_grad():
        # x = x.to(device) # Ensure input is on CPU
        # features = feature_extractor(x).cpu().numpy() # Keep output on CPU
        # f_all = np.vstack([f_all, features])
        # y_all = np.concatenate([y_all, y.numpy()]) # Ensure Labels are NumPy
        f_all = np.vstack([f_all, feature_extractor(x.cuda()).cpu()])
        y_all = np.concatenate([y_all, y])
    np.savez(filename, f_all=f_all, y_all=y_all)

```

Downloading: "https://github.com/pytorch/vision/zipball/v0.10.0" to /root/.cache/torch/hub/v0.10.0.zip

/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

warnings.warn(

/usr/local/lib/python3.11/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=VGG16\_Weights.IMAGENET1K\_V1`. You can also use `weights=VGG16\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.pth

100%|██████████| 528M/528M [00:03<00:00, 178MB/s]

100%|██████████| 58/58 [00:36<00:00, 1.58it/s]

QUESTION 20:

In a brief paragraph explain how the helper code base is performing feature extraction.

ANSWER:

The helper code loads the VGG-16 model (pretrained on ImageNet) but only uses the first portion of the model's layers, ensuring the output of high-level features. First, the input images are passed through convolutional layers which extract basic visual elements. A pooling layer is then applied to reduce the feature map size while preserving important information. Next, the 2D feature map is flattened to a single-dimensional feature vector. Finally, the first fully-connected layer of the VGG-16 model is applied which reduces the dimensionality of the vector into a 4096-dimensional feature vector.

QUESTION 21:

How many pixels are there in the original images? How many features does the VGG network extract per image; i.e what is the dimension of each feature vector for an image sample?

```
In [5]: print(f_all.shape, y_all.shape)
        num_features = f_all.shape[1]
```

```
(3670, 4096) (3670,)
```

ANSWER:

The original images vary slightly in resolution but are resized to 224x224x3 RGB images in the dataloader provided. The feature extractor then returns a flattened 4096-dimensional feature vector (for each image).

QUESTION 22:

Are the extracted features dense or sparse? (Compare with sparse TF-IDF features in text.)

```
In [6]: sparsity_ratio = np.mean(f_all == 0)
        print(f"Sparsity Ratio: {sparsity_ratio:.7f}")
```

```
Sparsity Ratio: 0.0000001
```

ANSWER:

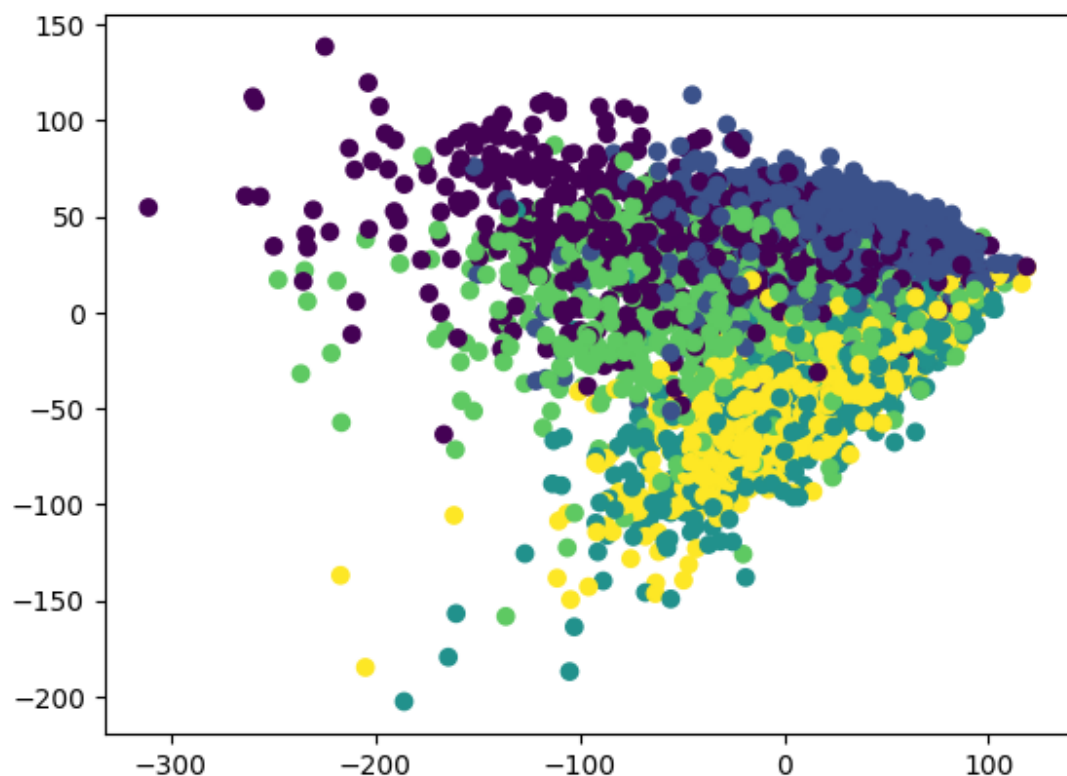
The extracted features are dense. The resultant feature vector is the output of the fully connected layer which contains meaningful (non-zero) information from the image. We can confirm this by checking the rate of non-zeroes in the feature vectors. As seen above, the ratio of zeroes to non-zeroes is essentially zero, confirming that these features are, in fact, dense.

QUESTION 23:

In order to inspect the high-dimensional features, t-SNE is a popular off-the-shelf choice for visualizing Vision features. Map the features you have extracted onto 2 dimensions with t-SNE. Then plot the mapped feature vectors along x and y axes. Color-code the data points with ground-truth labels. Describe your observation.

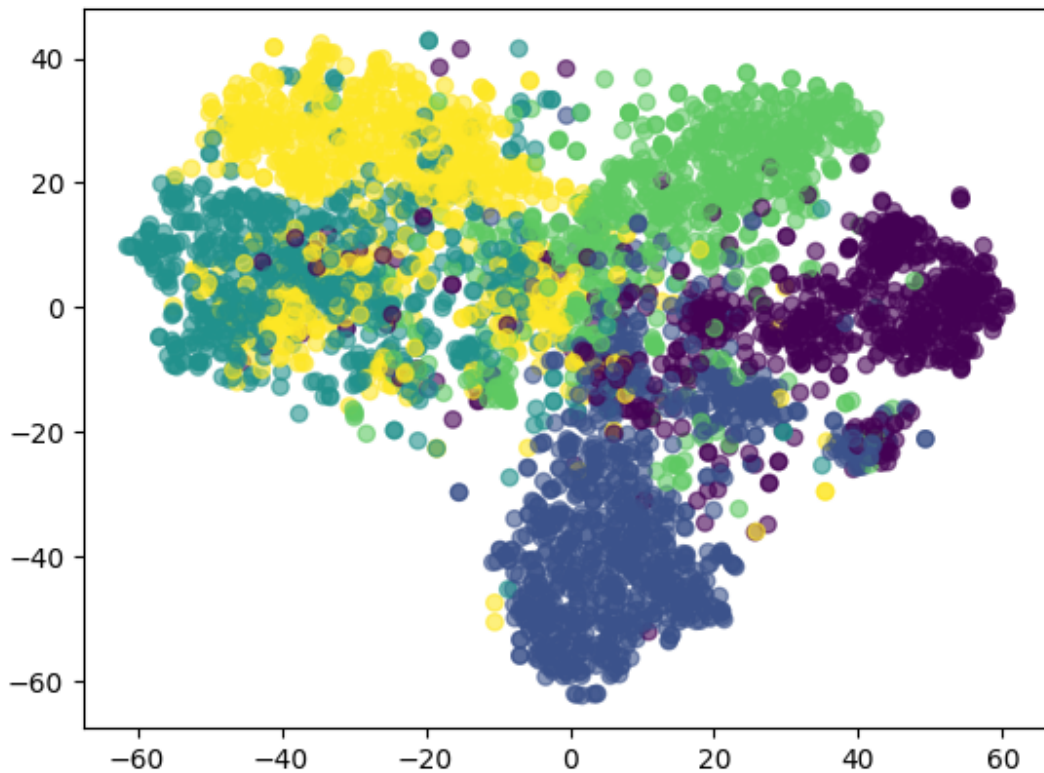
```
In [7]: f_pca = PCA(n_components=2).fit_transform(f_all)
plt.scatter(*f_pca.T, c=y_all)
```

Out[7]: <matplotlib.collections.PathCollection at 0x7bb2b2548410>



```
In [8]: from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, init='random', random_state=42)
f_tsne = tsne.fit_transform(f_all)
plt.scatter(f_tsne[:, 0], f_tsne[:, 1], c=y_all, alpha=0.6)
```

Out[8]: <matplotlib.collections.PathCollection at 0x7bb2b25482d0>



ANSWER:

Based on the visual representation of the features via t-SNE, the clusters are relatively well separated. The significantly worse separation offered by the PCA visualization suggests the non-linearity present in the dataset.

While PCA is a powerful method for dimensionality reduction, it is limited to “linear” transformations. This might not be particularly good if a dataset is distributed non-linearly. An alternative approach is use of an “autoencoder” or UMAP. The helper has implemented an autoencoder which is ready to use.

QUESTION 24:

Report the best result (in terms of rand score) within the table below. For HDBSCAN, introduce a conservative parameter grid over min cluster size and min samples.

Module	Alternatives	Hyperparameters
Dimensionality Reduction	None	N/A
	SVD	r = 50
	UMAP	n_components = 50
	Autoencoder	num_features = 50
Clustering	K-Means	k = 5

Module**Alternatives****Hyperparameters**Agglomerative Clustering `n_clusters = 5`HDBSCAN `Min_cluster_size` & `min_samples`

```
In [9]: # Autoencoder
class Autoencoder(torch.nn.Module, TransformerMixin):
    def __init__(self, n_components):
        super().__init__()
        self.n_components = n_components
        self.n_features = None # to be determined with data
        self.encoder = None
        self.decoder = None

    def _create_encoder(self):
        return nn.Sequential(
            nn.Linear(4096, 1280),
            nn.ReLU(True),
            nn.Linear(1280, 640),
            nn.ReLU(True), nn.Linear(640, 120), nn.ReLU(True), nn.Linear(120, self.

    def _create_decoder(self):
        return nn.Sequential(
            nn.Linear(self.n_components, 120),
            nn.ReLU(True),
            nn.Linear(120, 640),
            nn.ReLU(True),
            nn.Linear(640, 1280),
            nn.ReLU(True), nn.Linear(1280, 4096))

    def forward(self, X):
        encoded = self.encoder(X)
        decoded = self.decoder(encoded)
        return decoded

    def fit(self, X):
        X = torch.tensor(X, dtype=torch.float32, device='cuda')
        self.n_features = X.shape[1]
        self.encoder = self._create_encoder()
        self.decoder = self._create_decoder()
        self.cuda()
        self.train()

        criterion = nn.MSELoss()
        optimizer = torch.optim.Adam(self.parameters(), lr=1e-3, weight_decay=1e-5)

        dataset = TensorDataset(X)
        dataloader = DataLoader(dataset, batch_size=128, shuffle=True)

        for epoch in tqdm(range(100)):
            for (X_,) in dataloader:
                X_ = X_.cuda()
                # =====forward=====
                output = self(X_)
```

```

        loss = criterion(output, X_)
        # =====backward=====
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    return self

def transform(self, X):
    X = torch.tensor(X, dtype=torch.float32, device='cuda')
    self.eval()
    with torch.no_grad():
        return self.encoder(X).cpu().numpy()

```

```

In [10]: X_em = Autoencoder(n_components=2).fit_transform(f_all)
         plt.scatter(*X_em.T, c=y_all)

```

```

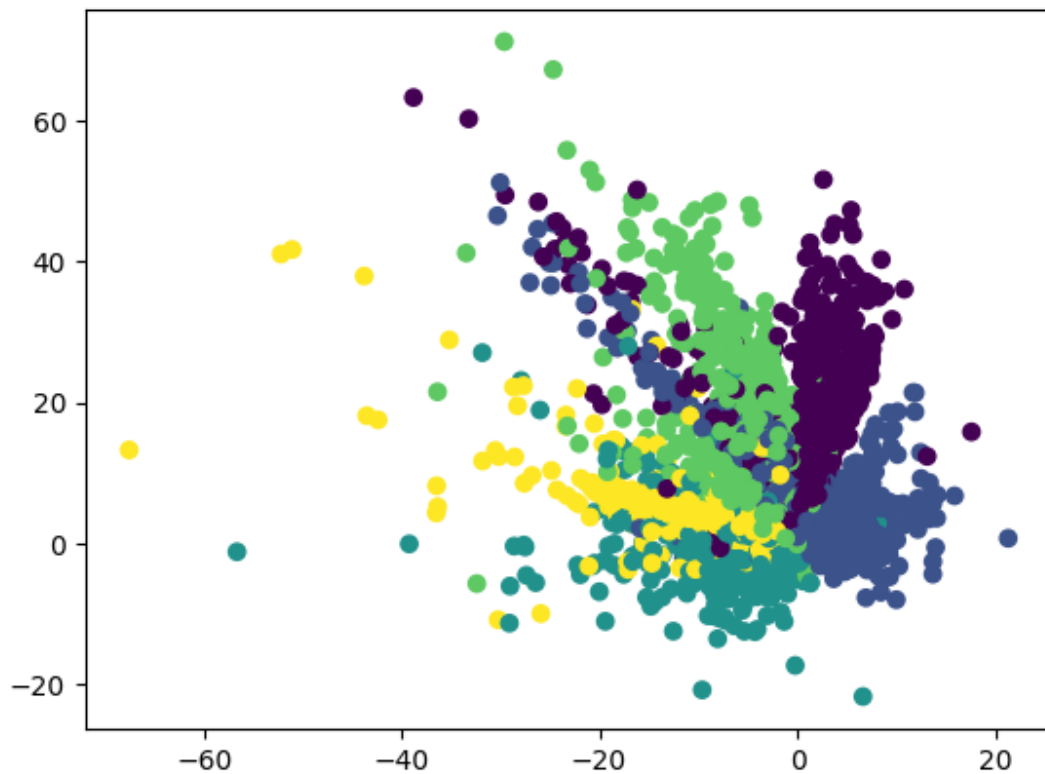
100%|██████████| 100/100 [00:19<00:00, 5.25it/s]

```

```

Out[10]: <matplotlib.collections.PathCollection at 0x7bb2a99e9890>

```



```

In [11]: # =====
         # Dimensionality Reduction Methods
         # =====

         # Autoencoder (n_components=50)
         f_autoencoder = Autoencoder(n_components=50).fit_transform(f_all)

         # SVD (r=50)
         f_svd = TruncatedSVD(n_components=50).fit_transform(f_all)
         # svd_features = svd.fit_transform(f_all)

```

```

# UMAP (n_components=50)
f_umap = UMAP(n_components=50).fit_transform(f_all)
# umap_features = umap.fit_transform(f_all)

# =====
# Clustering Methods
# =====

# K-means (k=5)
km_clustering = KMeans(n_clusters=5, random_state=0)

# Agglomerative clustering (n_clusters=5)
agg_clustering = AgglomerativeClustering(n_clusters=5)

```

100%|██████████| 100/100 [00:24<00:00, 4.16it/s]
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.

warnings.warn(

No DR

```

In [12]: # K-Means w/out D-Reduction
pred_km_none = km_clustering.fit_predict(f_all)
km_none_score = adjusted_rand_score(y_all, pred_km_none)
print("K-Means w/out D-Reduction Adjusted Rand score: ", km_none_score)

# Agglomerative w/out D-Reduction
pred_ac_none = agg_clustering.fit_predict(f_all)
ac_none_score = adjusted_rand_score(y_all, pred_ac_none)
print("Agglomerative w/out D-Reduction Adjusted Rand score: ", ac_none_score)

```

K-Means w/out D-Reduction Adjusted Rand score: 0.21134055272053653
Agglomerative w/out D-Reduction Adjusted Rand score: 0.2184499487113686

DR=SVD

```

In [13]: # SVD + K-Means
pred_svd_km = km_clustering.fit_predict(f_svd)
svd_km_score = adjusted_rand_score(y_all, pred_svd_km)
print("SVD + K-Means Adjusted Rand score: ", svd_km_score)

# SVD + Agglomerative
pred_svd_ac = agg_clustering.fit_predict(f_svd)
svd_ac_score = adjusted_rand_score(y_all, pred_svd_ac)
print("SVD + Agglomerative Adjusted Rand score: ", svd_ac_score)

```

SVD + K-Means Adjusted Rand score: 0.19916741660324128
SVD + Agglomerative Adjusted Rand score: 0.184963575854478

DR=UMAP

```

In [14]: # UMAP + K-Means
pred_umap_km = km_clustering.fit_predict(f_umap)
umap_km_score = adjusted_rand_score(y_all, pred_umap_km)
print("UMAP + K-Means Adjusted Rand score: ", umap_km_score)

```

```

# UMAP + Agglomerative
pred_umap_ac = agg_clustering.fit_predict(f_umap)
umap_ac_score = adjusted_rand_score(y_all, pred_umap_ac)
print("UMAP + Agglomerative Adjusted Rand score: ", umap_ac_score)

```

UMAP + K-Means Adjusted Rand score: 0.3970321079124104

UMAP + Agglomerative Adjusted Rand score: 0.36727688061403674

DR=Autoencoder

```

In [15]: # Autoencoder + K-Means
pred_auto_km = km_clustering.fit_predict(f_autoencoder)
auto_km_score = adjusted_rand_score(y_all, pred_auto_km)
print("Autoencoder + K-Means Adjusted Rand score: ", auto_km_score)

# Autoencoder + Agglomerative
pred_auto_ac = agg_clustering.fit_predict(f_autoencoder)
auto_ac_score = adjusted_rand_score(y_all, pred_auto_ac)
print("Autoencoder + Agglomerative Adjusted Rand score: ", auto_ac_score)

```

Autoencoder + K-Means Adjusted Rand score: 0.2333235266701561

Autoencoder + Agglomerative Adjusted Rand score: 0.21729199334227492

```

In [16]: # HDBSCAN
from hdbscan import HDBSCAN
from sklearn.model_selection import ParameterGrid

param_grid = {
    "min_cluster_size": [5, 10, 15],
    "min_samples": [1, 5, 10]
}

def get_best_hdbscan_params(param_grid, features):
    grid = list(ParameterGrid(param_grid)) # get all possible combos
    best_score = -1
    best_params = None

    for params in grid:
        clusterer = hdbscan.HDBSCAN(min_cluster_size=params["min_cluster_size"], mi
        labels = clusterer.fit_predict(features)
        score = adjusted_rand_score(y_all, labels)

        print(f"Params: {params}, Rand Score: {score:.4f}")

        # Keep track of best result
        if score > best_score:
            best_score = score
            best_params = params

    # Print tbest param
    print(f"Best Rand Score: {best_score:.4f} with Params: {best_params}")

print("No DR")
get_best_hdbscan_params(param_grid, f_all)
print("SVD")
get_best_hdbscan_params(param_grid, f_svd)

```

```
print("UMAP")
get_best_hdbscan_params(param_grid, f_umap)
print("Autoencoder")
get_best_hdbscan_params(param_grid, f_autoencoder)
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 1}, Rand Score: 0.0150
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 5}, Rand Score: 0.0067
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 10}, Rand Score: 0.0000
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 1}, Rand Score: 0.0150
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 5}, Rand Score: 0.0000
```

```

/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 10}, Rand Score: 0.0000
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 1}, Rand Score: 0.0137
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 5}, Rand Score: 0.0000
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 10}, Rand Score: 0.0000
Best Rand Score: 0.0150 with Params: {'min_cluster_size': 10, 'min_samples': 1}
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 1}, Rand Score: 0.0204
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

```
Params: {'min_cluster_size': 5, 'min_samples': 5}, Rand Score: 0.0055
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 10}, Rand Score: 0.0064
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 1}, Rand Score: 0.0161
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 5}, Rand Score: 0.0098
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 10}, Rand Score: 0.0000
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 1}, Rand Score: 0.0265
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

```
Params: {'min_cluster_size': 15, 'min_samples': 5}, Rand Score: 0.0098
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 10}, Rand Score: 0.0000
Best Rand Score: 0.0265 with Params: {'min_cluster_size': 15, 'min_samples': 1}
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 1}, Rand Score: 0.0163
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 5}, Rand Score: 0.0378
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 10}, Rand Score: 0.0022
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 1}, Rand Score: 0.0426
```

```

/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 5}, Rand Score: 0.0292
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 10}, Rand Score: 0.0022
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 1}, Rand Score: 0.0565
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 5}, Rand Score: 0.0394
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 10}, Rand Score: 0.0022
Best Rand Score: 0.0565 with Params: {'min_cluster_size': 15, 'min_samples': 1}
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(

```

```
Params: {'min_cluster_size': 5, 'min_samples': 1}, Rand Score: 0.0024
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 5}, Rand Score: 0.0300
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 5, 'min_samples': 10}, Rand Score: 0.0026
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 1}, Rand Score: 0.0257
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 5}, Rand Score: 0.0026
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 10, 'min_samples': 10}, Rand Score: 0.0000
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

```

Params: {'min_cluster_size': 15, 'min_samples': 1}, Rand Score: 0.0230
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 5}, Rand Score: 0.0000
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Params: {'min_cluster_size': 15, 'min_samples': 10}, Rand Score: 0.0000
Best Rand Score: 0.0300 with Params: {'min_cluster_size': 5, 'min_samples': 5}

```

ANSWER:

	None	SVD	UMAP	Autoencoder
K-Means	0.211	0.199	0.397	0.233
Agglomerative Clustering	0.218	0.1849	0.367	0.217
HDBSCAN	0.0150 {'min_cluster_size': 10, 'min_samples': 1}	0.0265 {'min_cluster_size': 15, 'min_samples': 1}	0.0565 {'min_cluster_size': 15, 'min_samples': 1}	0.0300 {'min_cluster_size': 5, 'min_samples': 5}

Lastly, we can conduct an experiment to ensure that VGG features are rich enough in information about the data classes. In particular, we can train a fully-connected neural network classifier to predict the labels of data. For this task, you may use the MLP4 module provided in the helper code base.

QUESTION 25:

Report the test accuracy of the MLP classifier on the original VGG features. Report the same when using the reduced-dimension features (you have freedom in choosing the dimensionality reduction algorithm and its parameters). Does the performance of the model suffer with the reduced-dimension representations? Is it significant? Does the success in classification make sense in the context of the clustering results obtained for the same features in Question 24

```

In [18]: class MLP(torch.nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(num_features, 1280),
            nn.ReLU(True),
            nn.Linear(1280, 640),
            nn.ReLU(True),
            nn.Linear(640, 5),
            nn.LogSoftmax(dim=1)
        )
        self.cuda()

    def forward(self, X):
        return self.model(X)

    def train(self, X, y):
        X = torch.tensor(X, dtype=torch.float32, device='cuda')
        y = torch.tensor(y, dtype=torch.int64, device='cuda')

        self.model.train()

        criterion = nn.NLLLoss()
        optimizer = torch.optim.Adam(self.parameters(), lr=1e-3, weight_decay=1e-5)

        dataset = TensorDataset(X, y)
        dataloader = DataLoader(dataset, batch_size=128, shuffle=True)

        for epoch in tqdm(range(100)):
            for (X_, y_) in dataloader:
                X_ = X_.cuda()
                y_ = y_.cuda()
                # =====forward=====
                output = self(X_)
                loss = criterion(output, y_)
                # =====backward=====
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()
            return self

    def eval(self, X_test, y_test):
        # Convert test data to torch tensors and move to CUDA
        X_test = torch.tensor(X_test, dtype=torch.float32, device='cuda')
        y_test = torch.tensor(y_test, dtype=torch.int64, device='cuda')

        self.model.eval() # Set model to evaluation mode

        with torch.no_grad(): # Disable gradient computation
            X_test = X_test.cuda()
            y_test = y_test.cuda()

            output = self(X_test)
            predictions = torch.argmax(output, dim=1)

```

```
accuracy = (predictions == y_test).float().mean().item()

return accuracy
```

```
In [22]: # Without Dimensionality Reduction
mlp = MLP(num_features=4096)
mlp.train(f_all[:2936], y_all[:2936])
mlp_score = mlp.eval(f_all[2936:], y_all[2936:])
print(f'MLP Evaluation Accuracy: {mlp_score:.4f}')
```

```
100%|██████████| 100/100 [00:14<00:00, 6.85it/s]
MLP Evaluation Accuracy: 0.9046
```

```
In [26]: # With Dimensionality Reduction (Autoencoder)
mlp_auto = MLP(num_features=50)
mlp_auto.train(f_autoencoder[:2936], y_all[:2936])
mlp_auto_score = mlp_auto.eval(f_autoencoder[2936:], y_all[2936:])
print(f'MLP (w/ DR) Evaluation Accuracy: {mlp_auto_score:.4f}')
```

```
100%|██████████| 100/100 [00:09<00:00, 10.01it/s]
MLP (w/ DR) Evaluation Accuracy: 0.8678
```

ANSWER:

I shuffled the data and split the data into 80% training, 20% testing. On the original VGG features the MLP model achieved a test accuracy of 90.46%. For comparison, I used the Autoencoder defined earlier for dimensionality reduction and achieved a test accuracy of 86.78%. Applying dimensionality reduction appears to have had a marginally negative impact on performance, with a drop in accuracy of 3.68%. This doesn't entirely line up with the clustering scores assigned in Q24, but the scores are close enough that it could be attributed to differences in feature representation

Part 3 - Clustering using both image and text

In part 1 and part 2, we have practiced the art of clustering text and images separately. However, can we map image and text to the same space? In the Pokemon world, Pokedex catalogs Pokemon's appearances and various metadata. We will build our Pokedex from image dataset link and meta metadata link. Fortunately, ECE 219 Gym kindly provides new Pokemon trainers with the helper code for data preprocessing and inferencing. Please find the code on Bruinlearn modules Week 4.

We will use the pre-trained CLIP [8] to illustrate the idea of multimodal clustering. CLIP (Contrastive Language–Image Pretraining) is an innovative model developed by OpenAI, designed to understand and connect concepts from both text and images. CLIP is trained on a vast array of internet-sourced text-image pairs. This extensive training enables the model to understand a broad spectrum of visual concepts and their textual descriptions.

CLIP consists of two primary components: a text encoder and an image encoder. The text encoder processes textual data, converting sentences and phrases into numerical representations. Simultaneously, the image encoder transforms visual inputs into a corresponding set of numerical values. These encoders are trained to map both text and images into a shared embedding space, allowing the model to compare and relate the two different types of data directly. The training employs a contrastive learning approach, where the model learns to match corresponding text and image pairs against numerous non-matching pairs. This approach helps the model in accurately associating images with their relevant textual descriptions and vice versa.

```
In [ ]: !pip install datasets transformers numpy pandas Pillow matplotlib
!pip install torch tqdm scipy
!pip install git+https://github.com/openai/CLIP.git
!pip install plotly umap-learn
```

Collecting datasets

Downloading datasets-3.2.0-py3-none-any.whl.metadata (20 kB)
 Requirement already satisfied: transformers in /usr/local/lib/python3.11/dist-packages (4.48.2)
 Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (1.26.4)
 Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (2.2.2)
 Requirement already satisfied: Pillow in /usr/local/lib/python3.11/dist-packages (11.1.0)
 Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (3.10.0)
 Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from datasets) (3.17.0)
 Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (17.0.0)
 Collecting dill<0.3.9,>=0.3.0 (from datasets)
 Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
 Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.11/dist-packages (from datasets) (2.32.3)
 Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.11/dist-packages (from datasets) (4.67.1)
 Collecting xxhash (from datasets)
 Downloading xxhash-3.5.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
 Collecting multiprocess<0.70.17 (from datasets)
 Downloading multiprocess-0.70.16-py311-none-any.whl.metadata (7.2 kB)
 Collecting fsspec<=2024.9.0,>=2023.1.0 (from fsspec[http]<=2024.9.0,>=2023.1.0->datasets)
 Downloading fsspec-2024.9.0-py3-none-any.whl.metadata (11 kB)
 Requirement already satisfied: aiohttp in /usr/local/lib/python3.11/dist-packages (from datasets) (3.11.11)
 Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (0.28.1)
 Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from datasets) (24.2)
 Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from datasets) (6.0.2)
 Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (2024.11.6)
 Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.21.0)
 Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.5.2)
 Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas) (2.8.2)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.1)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.1)
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.1)
 Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (4.55.8)

```

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.8)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.1)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (2.4.4)
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.3.2)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (0.2.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.18.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.23.0->datasets) (4.12.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests>=2.32.2->datasets) (2025.1.31)
Downloading datasets-3.2.0-py3-none-any.whl (480 kB)
_____ 480.6/480.6 kB 7.8 MB/s eta 0:00
0:00
Downloading dill-0.3.8-py3-none-any.whl (116 kB)
_____ 116.3/116.3 kB 9.7 MB/s eta 0:00
0:00
Downloading fsspec-2024.9.0-py3-none-any.whl (179 kB)
_____ 179.3/179.3 kB 13.7 MB/s eta 0:00
0:00
Downloading multiprocess-0.70.16-py311-none-any.whl (143 kB)
_____ 143.5/143.5 kB 5.2 MB/s eta 0:00
0:00
Downloading xxhash-3.5.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (194 kB)
_____ 194.8/194.8 kB 7.3 MB/s eta 0:00
0:00
Installing collected packages: xxhash, fsspec, dill, multiprocess, datasets
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2024.10.0
    Uninstalling fsspec-2024.10.0:
      Successfully uninstalled fsspec-2024.10.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
gcsfs 2024.10.0 requires fsspec==2024.10.0, but you have fsspec 2024.9.0 which is incompatible.

```

```

torch 2.5.1+cu124 requires nvidia-cublas-cu12==12.4.5.8; platform_system ==
"Linux" and platform_machine == "x86_64", but you have nvidia-cublas-cu12 1
2.5.3.2 which is incompatible.
torch 2.5.1+cu124 requires nvidia-cuda-cupti-cu12==12.4.127; platform_system
== "Linux" and platform_machine == "x86_64", but you have nvidia-cuda-cupti-
cu12 12.5.82 which is incompatible.
torch 2.5.1+cu124 requires nvidia-cuda-nvrtc-cu12==12.4.127; platform_system
== "Linux" and platform_machine == "x86_64", but you have nvidia-cuda-nvrtc-
cu12 12.5.82 which is incompatible.
torch 2.5.1+cu124 requires nvidia-cuda-runtime-cu12==12.4.127; platform_syst
em == "Linux" and platform_machine == "x86_64", but you have nvidia-cuda-run
time-cu12 12.5.82 which is incompatible.
torch 2.5.1+cu124 requires nvidia-cudnn-cu12==9.1.0.70; platform_system ==
"Linux" and platform_machine == "x86_64", but you have nvidia-cudnn-cu12 9.
3.0.75 which is incompatible.
torch 2.5.1+cu124 requires nvidia-cufft-cu12==11.2.1.3; platform_system ==
"Linux" and platform_machine == "x86_64", but you have nvidia-cufft-cu12 11.
2.3.61 which is incompatible.
torch 2.5.1+cu124 requires nvidia-curand-cu12==10.3.5.147; platform_system =
= "Linux" and platform_machine == "x86_64", but you have nvidia-curand-cu12
10.3.6.82 which is incompatible.
torch 2.5.1+cu124 requires nvidia-cusolver-cu12==11.6.1.9; platform_system =
= "Linux" and platform_machine == "x86_64", but you have nvidia-cusolver-cu1
2 11.6.3.83 which is incompatible.
torch 2.5.1+cu124 requires nvidia-cuspars-cu12==12.3.1.170; platform_system
== "Linux" and platform_machine == "x86_64", but you have nvidia-cuspars-cu
12 12.5.1.3 which is incompatible.
torch 2.5.1+cu124 requires nvidia-nvjitlink-cu12==12.4.127; platform_system
== "Linux" and platform_machine == "x86_64", but you have nvidia-nvjitlink-c
u12 12.5.82 which is incompatible.
Successfully installed datasets-3.2.0 dill-0.3.8 fsspec-2024.9.0 multiproces
s-0.70.16 xxhash-3.5.0
Requirement already satisfied: torch in /usr/local/lib/python3.11/dist-packa
ges (2.5.1+cu124)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packag
es (4.67.1)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packa
ges (1.13.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-pa
ckages (from torch) (3.17.0)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/py
thon3.11/dist-packages (from torch) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-pa
ckages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-pack
ages (from torch) (3.1.5)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-pack
ages (from torch) (2024.9.0)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.
whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_6
4.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.

```

```

whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl.m
etadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl.
metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl.m
etadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.5.147 (from torch)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.wh
l.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.wh
l.metadata (1.6 kB)
Collecting nvidia-cuspars-cu12==12.3.1.170 (from torch)
  Downloading nvidia_cuspars-cu12-12.3.1.170-py3-none-manylinux2014_x86_64.
whl.metadata (1.6 kB)
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/py
thon3.11/dist-packages (from torch) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /usr/local/lib/
python3.11/dist-packages (from torch) (12.4.127)
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch)
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.w
hl.metadata (1.5 kB)
Requirement already satisfied: triton==3.1.0 in /usr/local/lib/python3.11/di
st-packages (from torch) (3.1.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/di
st-packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.
11/dist-packages (from sympy==1.13.1->torch) (1.3.0)
Requirement already satisfied: numpy<2.3,>=1.22.4 in /usr/local/lib/python3.
11/dist-packages (from scipy) (1.26.4)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/
dist-packages (from jinja2->torch) (3.0.2)
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (3
63.4 MB)
  _____ 363.4/363.4 MB 4.3 MB/s eta 0:0
0:00
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.wh
l (13.8 MB)
  _____ 13.8/13.8 MB 92.4 MB/s eta 0:00:
00
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.wh
l (24.6 MB)
  _____ 24.6/24.6 MB 67.7 MB/s eta 0:00:
00
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.
whl (883 kB)
  _____ 883.7/883.7 kB 51.8 MB/s eta 0:0
0:00
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (66
4.8 MB)
  _____ 664.8/664.8 MB 2.2 MB/s eta 0:0
0:00

```

Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (211.5 MB)

211.5/211.5 MB 6.0 MB/s eta 0:00:00

Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl (56.3 MB)

56.3/56.3 MB 11.3 MB/s eta 0:00:00

Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl (127.9 MB)

127.9/127.9 MB 7.9 MB/s eta 0:00:00

Downloading nvidia_cusparsel_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl (207.5 MB)

207.5/207.5 MB 5.6 MB/s eta 0:00:00

Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (21.1 MB)

21.1/21.1 MB 59.0 MB/s eta 0:00:00

Installing collected packages: nvidia-nvjitlink-cu12, nvidia-curand-cu12, nvidia-cufft-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, nvidia-cusparsel-cu12, nvidia-cudnn-cu12, nvidia-cusolver-cu12

Attempting uninstall: nvidia-nvjitlink-cu12

Found existing installation: nvidia-nvjitlink-cu12 12.5.82

Uninstalling nvidia-nvjitlink-cu12-12.5.82:

Successfully uninstalled nvidia-nvjitlink-cu12-12.5.82

Attempting uninstall: nvidia-curand-cu12

Found existing installation: nvidia-curand-cu12 10.3.6.82

Uninstalling nvidia-curand-cu12-10.3.6.82:

Successfully uninstalled nvidia-curand-cu12-10.3.6.82

Attempting uninstall: nvidia-cufft-cu12

Found existing installation: nvidia-cufft-cu12 11.2.3.61

Uninstalling nvidia-cufft-cu12-11.2.3.61:

Successfully uninstalled nvidia-cufft-cu12-11.2.3.61

Attempting uninstall: nvidia-cuda-runtime-cu12

Found existing installation: nvidia-cuda-runtime-cu12 12.5.82

Uninstalling nvidia-cuda-runtime-cu12-12.5.82:

Successfully uninstalled nvidia-cuda-runtime-cu12-12.5.82

Attempting uninstall: nvidia-cuda-nvrtc-cu12

Found existing installation: nvidia-cuda-nvrtc-cu12 12.5.82

Uninstalling nvidia-cuda-nvrtc-cu12-12.5.82:

Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.5.82

Attempting uninstall: nvidia-cuda-cupti-cu12

Found existing installation: nvidia-cuda-cupti-cu12 12.5.82

Uninstalling nvidia-cuda-cupti-cu12-12.5.82:

Successfully uninstalled nvidia-cuda-cupti-cu12-12.5.82

Attempting uninstall: nvidia-cublas-cu12

Found existing installation: nvidia-cublas-cu12 12.5.3.2

Uninstalling nvidia-cublas-cu12-12.5.3.2:

Successfully uninstalled nvidia-cublas-cu12-12.5.3.2

Attempting uninstall: nvidia-cusparsel-cu12

Found existing installation: nvidia-cusparsel-cu12 12.5.1.3

Uninstalling nvidia-cusparsel-cu12-12.5.1.3:

Successfully uninstalled nvidia-cusparsel-cu12-12.5.1.3

```

Attempting uninstall: nvidia-cudnn-cu12
  Found existing installation: nvidia-cudnn-cu12 9.3.0.75
  Uninstalling nvidia-cudnn-cu12-9.3.0.75:
    Successfully uninstalled nvidia-cudnn-cu12-9.3.0.75
Attempting uninstall: nvidia-cusolver-cu12
  Found existing installation: nvidia-cusolver-cu12 11.6.3.83
  Uninstalling nvidia-cusolver-cu12-11.6.3.83:
    Successfully uninstalled nvidia-cusolver-cu12-11.6.3.83
Successfully installed nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-cu12-12.4.127 nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3 nvidia-curand-cu12-10.3.5.147 nvidia-cusolver-cu12-11.6.1.9 nvidia-cuspars-cu12-12.3.1.170 nvidia-nvjitlink-cu12-12.4.127
Collecting git+https://github.com/openai/CLIP.git
  Cloning https://github.com/openai/CLIP.git to /tmp/pip-req-build-b1l2ltc_
  Running command git clone --filter=blob:none --quiet https://github.com/openai/CLIP.git /tmp/pip-req-build-b1l2ltc_
  Resolved https://github.com/openai/CLIP.git to commit dcba3cb2e2827b402d2701e7e1c7d9fed8a20ef1
  Preparing metadata (setup.py) ... done
Collecting ftfy (from clip==1.0)
  Downloading ftfy-6.3.1-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from clip==1.0) (24.2)
Requirement already satisfied: regex in /usr/local/lib/python3.11/dist-packages (from clip==1.0) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from clip==1.0) (4.67.1)
Requirement already satisfied: torch in /usr/local/lib/python3.11/dist-packages (from clip==1.0) (2.5.1+cu124)
Requirement already satisfied: torchvision in /usr/local/lib/python3.11/dist-packages (from clip==1.0) (0.20.1+cu124)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-packages (from ftfy->clip==1.0) (0.2.13)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (3.17.0)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (4.12.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (3.1.5)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (2024.9.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (12.4.127)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (12.4.127)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (12.4.127)
Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (9.1.0.70)
Requirement already satisfied: nvidia-cublas-cu12==12.4.5.8 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (12.4.5.8)
Requirement already satisfied: nvidia-cufft-cu12==11.2.1.3 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (11.2.1.3)

```

Requirement already satisfied: nvidia-curand-cu12==10.3.5.147 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (10.3.5.147)
 Requirement already satisfied: nvidia-cusolver-cu12==11.6.1.9 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (11.6.1.9)
 Requirement already satisfied: nvidia-cuspars-cu12==12.3.1.170 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (12.3.1.170)
 Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (2.21.5)
 Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (12.4.127)
 Requirement already satisfied: nvidia-nvjitlink-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (12.4.127)
 Requirement already satisfied: triton==3.1.0 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (3.1.0)
 Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packages (from torch->clip==1.0) (1.13.1)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy==1.13.1->torch->clip==1.0) (1.3.0)
 Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from torchvision->clip==1.0) (1.26.4)
 Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.11/dist-packages (from torchvision->clip==1.0) (11.1.0)
 Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->torch->clip==1.0) (3.0.2)
 Downloading ftfy-6.3.1-py3-none-any.whl (44 kB)

44.8/44.8 kB 2.9 MB/s eta 0:00:00

Building wheels for collected packages: clip
 Building wheel for clip (setup.py) ... done
 Created wheel for clip: filename=clip-1.0-py3-none-any.whl size=1369489 sha256=d1468ef5816fc7f208b62055e2778f0f05c84faa93889c615c5f0decca3faedc
 Stored in directory: /tmp/pip-ephem-wheel-cache-zj7e6hlh/wheels/3f/7c/a4/9b490845988bf7a4db33674d52f709f088f64392063872eb9a
 Successfully built clip
 Installing collected packages: ftfy, clip
 Successfully installed clip-1.0 ftfy-6.3.1
 Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (5.24.1)
 Collecting umap-learn
 Downloading umap_learn-0.5.7-py3-none-any.whl.metadata (21 kB)
 Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly) (9.0.0)
 Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from plotly) (24.2)
 Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (1.26.4)
 Requirement already satisfied: scipy>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (1.13.1)
 Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (1.6.1)
 Requirement already satisfied: numba>=0.51.2 in /usr/local/lib/python3.11/dist-packages (from umap-learn) (0.61.0)
 Collecting pynndescent>=0.5 (from umap-learn)
 Downloading pynndescent-0.5.13-py3-none-any.whl.metadata (6.8 kB)
 Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from umap-learn) (4.67.1)

Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba>=0.51.2->umap-learn) (0.44.0)
 Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.11/dist-packages (from pynndescent>=0.5->umap-learn) (1.4.2)
 Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.22->umap-learn) (3.5.0)
 Downloading umap_learn-0.5.7-py3-none-any.whl (88 kB)

88.8/88.8 kB 5.4 MB/s eta 0:00:00

Downloading pynndescent-0.5.13-py3-none-any.whl (56 kB)

56.9/56.9 kB 5.1 MB/s eta 0:00:00

Installing collected packages: pynndescent, umap-learn
 Successfully installed pynndescent-0.5.13 umap-learn-0.5.7

In []: *### QUESTION 28:*

In the first **and** second question, we investigated how CLIP creates 'clusters mapping images **and** texts of various Pokemon into a high-dimensional space **and** hood of these items **in** this space. For this question, please use t-SNE to visualize specifically **for** Pokemon types Bug, Fire, **and** Grass. You can use scatter plot with plotly. For the visualization, color-code each point based on its first type argument, **and** label each point **with** the Pokemon's name **and** types using 'hover'. This will enable you to identify each Pokemon represented **in** your visualization. After visualization, analyze it **and** discuss whether the clustering of Pokemon type

Mounted at /content/drive

In []: *# Code for Q28*

In []: After completing the visualization, analyze it **and** discuss whether the clustering

Answer:

In []: *# Code for Q26*
Load the Pokedex data
 pokedex = construct_pokedex()

In []: *# Define type queries*
 type_queries = {
 "Bug": "A Pokémon of Bug type",
 "Fire": "A Pokémon of Fire type",
 "Grass": "A Pokémon of Grass type",
 "Dark": "A Pokémon of Dark type",
 "Dragon": "A Pokémon of Dragon type"
 }

In []: **import** numpy **as** np

Load CLIP model
 model, preprocess, device = load_clip_model()

Encode text queries
 text_embeddings = clip_inference_text(model, preprocess, list(type_queries.values()))

```
# Encode Pokémon images
image_embeddings = clip_inference_image(model, preprocess, pokedex["image_pa

# Compute similarity between text queries and images
similarities = compute_similarity_text_to_image(image_embeddings, text_embed

# Get the top 5 most relevant Pokémon for each type
top_k_indices = np.argsort(similarities, axis=0)[-5:][::-1]
```

```
100%|████████████████████████████████████████████████████████████████████████████████| 890M/890M [00:21<00:00, 43.6MiB/s]
100%|████████████████████████████████████████████████████████████████████████████████| 754/754 [05:25<00:00, 2.31it/s]
```

```
In [ ]: # Function to plot top Pokémon results
def plot_top_pokemon(top_indices, query, type_name):
    fig, axes = plt.subplots(1, 5, figsize=(20, 5))
    fig.suptitle(f"Text Query: {query}", fontsize=16)

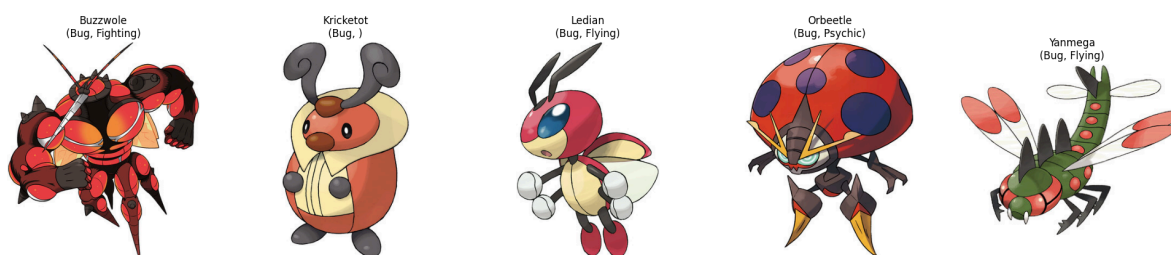
    for i, idx in enumerate(top_indices):
        pokemon_name = pokedex.iloc[idx]["Name"]
        pokemon_type1 = pokedex.iloc[idx]["Type1"]
        pokemon_type2 = pokedex.iloc[idx]["Type2"]
        img_path = pokedex.iloc[idx]["image_path"]

        # Load and display the Pokémon image
        image = Image.open(img_path)
        axes[i].imshow(image)
        axes[i].axis("off")
        axes[i].set_title(f"{pokemon_name}\n({pokemon_type1}, {pokemon_type2})")

    plt.tight_layout(pad=2.0)
    plt.show()
```

```
In [ ]: # Loop through all types and plot results
for type_name, query in type_queries.items():
    top_indices = top_k_indices[:, list(type_queries.keys()).index(type_name)]
    plot_top_pokemon(top_indices, query, type_name)
```

Text Query: A Pokémon of Bug type



Text Query: A Pokémon of Fire type



Text Query: A Pokémon of Grass type



Text Query: A Pokémon of Dark type



Text Query: A Pokémon of Dragon type



Answer

The results of the queries can be seen above. Judging by these results, the queries were actually very effective. In almost every single case the query resulted in pokemen from the rolodex that perfectly matched the query type. There are a few off cases where the pokemon is not of the correct type. For example, pokemon of bug, and fire were 100% accurate. This is most likwly because it prioritized the respective bright green and red colors which are. abit easier to isolate within images. One case where it didnt perform as well is for dark type. The queue seems to be . mit more ambiguous and the results reflect this. The pokemon are . bit darker in hugh but not always the correct type. This shows how ambiguity in queue and uniqueness can result in worst accuracy and results overall.

QUESTION 27:

Randomly select 10 Pokemon images from the dataset and use CLIP to find the most relevant types (use your preferred template, e.g "type: Bug"). For each selected Pokemon, please plot it and indicate:

- its name and first and second type;
- the five most relevant types predicted by CLIP and their predicted probabilities.

```
In [ ]: # Code for Q27
# Extract unique types
unique_types = pd.unique(pd.concat([pokedex['Type1'], pokedex['Type2']])).to

# Construct query dictionary
type_queries = {t: f"type: {t}" for t in unique_types if pd.notna(t)}

# Load CLIP model
model, preprocess, device = load_clip_model()

# Encode type queries using CLIP
query_embeddings = clip_inference_text(model, preprocess, list(type_queries.

# Select 10 random Pokémon (random seed = 108)
selected_pokemon = pokedex.sample(n=10, random_state=108).reset_index(drop=T
```

```
In [ ]: # Function to predict top 5 Pokémon types using CLIP
def predict_types(image_paths):
    # Encode Pokémon images using CLIP
    image_embeddings = clip_inference_image(model, preprocess, image_paths,

    # Compute similarity scores (text-to-image)
    similarity_scores = compute_similarity_text_to_image(image_embeddings, c

    # Get top 5 predicted types for each image
    predicted_types = []
    for scores in similarity_scores:
        top_indices = np.argsort(scores)[::-1][:5]
        relevant_types = [(list(type_queries.keys())[i], scores[i]) for i in
        predicted_types.append(relevant_types)

    return predicted_types

# Function to plot Pokémon with actual and predicted types
def plot_pokemon_with_predictions(pokemon_name, type1, type2, image_path, pr
    plt.figure(figsize=(6, 6))

    # Load and display
    image = Image.open(image_path)
    plt.imshow(image)
    plt.axis("off")

    # Title
    plt.title(f"{pokemon_name}\n Type 1: {type1}, Type 2: {type2}", fontsize

    # Display predicted types with probabilities
    caption = "\n".join([f"{ptype}: {prob:.4f}" for ptype, prob in predicted
    plt.figtext(0.5, 0.01, caption, wrap=True, horizontalalignment='center',

    plt.show()
```

```
In [ ]: # Process each Pokémon and display results
for _, row in selected_pokemon.iterrows():
    pokemon_name = row['Name']
    type1 = row['Type1']
```

```
type2 = row['Type2'] if pd.notna(row['Type2']) else "None"  
image_path = row['image_path']  
  
predicted_types = predict_types([image_path])[0] # Get predictions for  
plot_pokemon_with_predictions(pokemon_name, type1, type2, image_path, pr
```

100%|██████████| 1/1 [00:00<00:00, 32.84it/s]

Delphox
Type 1: Fire, Type 2: Psychic



Fire: 0.8140
: 0.0336
Electric: 0.0287
Psychic: 0.0254
Grass: 0.0220

100%|██████████| 1/1 [00:00<00:00, 27.22it/s]

Carbink
Type 1: Rock, Type 2: Fairy



Ice: 0.5020
Rock: 0.4788
Ground: 0.0089
Water: 0.0059
: 0.0016

100% | 1/1 [00:00<00:00, 26.55it/s]

Mew
Type 1: Psychic, Type 2:



Poison: 0.4900
Psychic: 0.1449
Electric: 0.0851
: 0.0567
Fighting: 0.0500

100%|██████████| 1/1 [00:00<00:00, 28.09it/s]

Simisage
Type 1: Grass, Type 2:



Grass: 0.8921
Ice: 0.0238
Electric: 0.0144
Poison: 0.0119
: 0.0112

100%|██████████| 1/1 [00:00<00:00, 22.80it/s]

Honedge
Type 1: Steel, Type 2: Ghost



Water: 0.4849
Psychic: 0.1599
Dark: 0.0710
Ghost: 0.0386
Steel: 0.0374

Sobble
Type 1: Water, Type 2:



Water: 0.6348
Ice: 0.0845
Electric: 0.0690
Fighting: 0.0358
Psychic: 0.0337

100%|██████████| 1/1 [00:00<00:00, 30.56it/s]

Lugia

Type 1: Psychic, Type 2: Flying



Water: 0.3005
Ice: 0.2693
Flying: 0.1334
Dragon: 0.0861
: 0.0530

100% | 1/1 [00:00<00:00, 30.62it/s]

Nihilego

Type 1: Rock, Type 2: Poison



Water: 0.5625
Ice: 0.1423
Ghost: 0.1337
Psychic: 0.0448
: 0.0323

100%|██████████| 1/1 [00:00<00:00, 27.43it/s]

Seel
Type 1: Water, Type 2:



Water: 0.5415
: 0.1677
Ice: 0.1083
Ghost: 0.0369
Psychic: 0.0357

100% | 1/1 [00:00<00:00, 25.65it/s]

Bulbasaur

Type 1: Grass, Type 2: Poison



Grass: 0.0437
 Water: 0.0847
 : 0.0506
 Poison: 0.0302
 Electric: 0.0232

QUESTION 28:

In the first and second question, we investigated how CLIP creates 'clusters' by mapping images and texts of various Pokemon into a high-dimensional space and explored neighbor- hood of these items in this space. For this question, please use t-SNE to visualize image clusters, specifically for Pokemon types Bug, Fire, and Grass. You can use scatter plot from python package plotly. For the visualization, color-code each point based on its first type type 1 using the 'color' argument, and label each point with the Pokemon's name and types using 'hover name'. This will enable you to identify each Pokemon represented in your visualization. After completing the visualization, analyze it and discuss whether the clustering of Pokemon types make sense to you.

```
In [ ]: # Code for Q28
# Select Pokémon of types Bug, Fire, and Grass
selected_types = ["Bug", "Fire", "Grass"]
filtered_pokedex = pokedex[pokedex["Type1"].isin(selected_types)].reset_index
```

```
# Load CLIP model
model, preprocess, device = load_clip_model()

# Compute image embeddings using CLIP
image_embeddings = clip_inference_image(model, preprocess, filtered_pokedex[

# Reduce dimensions using t-SNE
tsne = TSNE(n_components=2, perplexity=15, random_state=42, metric="cosine")
tsne_results = tsne.fit_transform(image_embeddings)

plot_data = pd.DataFrame({
    "x": tsne_results[:, 0],
    "y": tsne_results[:, 1],
    "Name": filtered_pokedex["Name"],
    "Type1": filtered_pokedex["Type1"],
    "Type2": filtered_pokedex["Type2"]
})

# Create plot
fig = px.scatter(
    plot_data,
    x="x",
    y="y",
    color="Type1",
    hover_data={"Name": True, "Type1": True, "Type2": True}
)

# Show plot
fig.show()
```

100%|██████████| 192/192 [00:04<00:00, 42.88it/s]

Code for Q26

Load the Pokedex data

```
pokedex = construct_pokedex()
```

QUESTION 27:

Randomly select 10 Pokemon images from the dataset and use CLIP to find the most relevant types (use your preferred template, e.g "type: Bug"). For each selected Pokemon, please plot it and indicate:

- its name and first and second type;
- the five most relevant types predicted by CLIP and their predicted probabilities.

